

EmpName :Rishi Kesava Vigneshver Manisekaran

EmpId :12575

Project Title : Online Salon Management System

Q15) Online Salon Management System

Online Salon & Spa Management web application, enables salon owners to automate the front-desk operation & daily activities such as Appointment Scheduling, Online Booking, Billing, tracking customers, Staff members, Inventory tracking, and others.

Tasks:

1. Perform W3H analysis on the given Case Study

PROJECT: Online Salon Management System	
What?	How?
What are the Modules used in the Online Salon Management System ? 1. Admin 2. User 3. Appointment Scheduling 4. Online Booking 5. Billing 6. Tracking Customers 7. Staff Members 8. Inventory Tracking	1.Admin a. Admin can Login Method 1: Login through Name and password Method 2: Login through Email id and password Method 3: Login through Social profiles b. Admin can add Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking Method 1: Add the Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking directly in the MySQL table using MVC through API Method 2: Add the Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking through the Excel Method 3: Add the Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking through the UI c. Admin can delete the Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking Method 1: Delete the Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking by Name Method 2: Delete the Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking by category Method 3: Delete the Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking by Id d. Admin can Edit the Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking Method 1: Edit the Appointment Scheduling, Online
1.What are the functionalities in the Admin module? a) Admin can Login b) Admin can add the Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking c) Admin can delete the Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking d) Admin can Edit the Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking e) Admin can view the Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking	
2.What are the functionalities in the User module? a) User can Login b) User can add the Appointment Scheduling ,Online Booking c) User can delete the Appointment Scheduling ,Online Booking	

d) User can Edit the Appointment Scheduling ,Online Booking

e) User can view the Appointment Scheduling ,Online Booking

3.What are the functionalities in the Appointment Scheduling module?

a) User can add the Service, Staff, Date and Time

b) User can delete the Service, Staff, Date and Time

c) User can Edit the Service, Staff, Date and Time

d) User can view the Service, Staff, Date and Time

4.What are the functionalities in the Online Booking module?

a) User can add the Online Booking

b) User can delete the Online Booking

c) User can Edit the Online Booking

d) User can view the Online Booking

5.What are the functionalities in the Billing module?

a) User can add the Billing

b) User can delete the Billing

c) User can Edit the Billing

d) User can view the Billing

6.What are the functionalities in the Tracking Customers module?

a) Admin can add the tracking customer details

b) Admin can delete the tracking customer details

c) Admin can Edit the tracking customer details

d) Admin can view the tracking customer details

7.What are the functionalities in the Staff Members module?

a) Admin can add Staff Name, Email, Phone, Role, Salon

b) Admin can delete Staff Name, Email, Phone, Role, Salon

Booking, Billing, tracking customers, staff Members, Inventory Tracking by Name

Method 2: Edit the Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking by category

Method 3: Edit the Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking by Id

e. Admin can view the Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking

Method 1: view the Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking by Name

Method 2: view the Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking by category

Method 3: view the Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking by Id

2.User

a. User can Login

Method 1: Login through Name and password

Method 2: Login through Email id and password

Method 3: Login through Social profiles

b. User can add Appointment Scheduling ,Online Booking

Method 1: Add the Appointment Scheduling ,Online Booking directly in the MySQL table using MVC through API

Method 2: Add the Appointment Scheduling ,Online Booking through the Excel

Method 3: Add the Appointment Scheduling ,Online Booking through the UI

c. User can delete the Appointment Scheduling ,Online Booking

Method 1: Delete the Appointment Scheduling ,Online Booking by Name

c) Admin can Edit Staff Name, Email, Phone, Role, Salon

d) Admin can view Staff Name, Email, Phone, Role, Salon

8.What are the functionalities in the Inventory Tracking module?

a) Admin can add Inventory Items, Quantities

b) Admin can delete Inventory Items, Quantities

c) Admin can Edit Inventory Items, Quantities

d) Admin can view Inventory Items, Quantities

9.What are the database tables required?

a) tbl_AdminLog

b) tbl_Appointments

c) tbl_Customers

d) tbl_Inventories

e) tbl_Salons

f) tbl_Services

g) tbl_Staffs

Method 2: Delete the Appointment Scheduling ,Online Booking by category

Method 3: Delete the Appointment Scheduling ,Online Booking by Id

d. User can Edit the Appointment Scheduling ,Online Booking

Method 1: Edit the Appointment Scheduling ,Online Booking by Name

Method 2: Edit the Appointment Scheduling ,Online Booking by category

Method 3: Edit the Appointment Scheduling ,Online Booking by Id

e. User can view the Appointment Scheduling ,Online Booking

Method 1: view the Appointment Scheduling ,Online Booking by Name

Method 2: view the Appointment Scheduling ,Online Booking by category

Method 3: view the Appointment Scheduling ,Online Booking by Id

3.Appointment Scheduling

a. User can add Service, Staff, Date and Time

Method 1: Add the Service, Staff, Date and Time directly in the MySQL table using MVC through API

Method 2: Add the Service, Staff, Date and Time through the Excel

Method 3: Add the Service, Staff, Date and Time through the UI

b. User can delete the Service, Staff, Date and Time

Method 1: Delete the Service, Staff, Date and Time by Name

Method 2: Delete the Service, Staff, Date and Time by category

Method 3: Delete the Service, Staff, Date and Time by Id

c. User can Edit the Service, Staff, Date and Time

Method 1: Edit the Service, Staff, Date and Time by Name

Method 2: Edit the Service, Staff, Date and Time by

category

Method 3: Edit the Service, Staff, Date and Time by Id

d. User can view the Service, Staff, Date and Time

Method 1: view the Service, Staff, Date and Time by Name

Method 2: view the Service, Staff, Date and Time by category

Method 3: View the Service, Staff, Date and Time by Id

4. Online Booking

a. User can add Online Booking

Method 1: Add the Online Booking directly in the MySQL table using MVC through API

Method 2: Add the Online Booking through the Excel

Method 3: Add the Online Booking through the UI

b. User can delete the Online Booking

Method 1: Delete the Online Booking by Name

Method 2: Delete the Online Booking by category

Method 3: Delete the Online Booking by Id

c. User can Edit the Online Booking

Method 1: Edit the Online Booking by Name

Method 2: Edit the Online Booking by category

Method 3: Edit the Online Booking by Id

d. User can view the Online Booking

Method 1: view the Online Booking by Name

Method 2: view the Online Booking by category

Method 3: View the Online Booking by Id

5. Billing module

a. Admin can add Billing

Method 1: Add the Billing directly in the MySQL table using MVC through API

Method 2: Add the Billing through the Excel

Method 3: Add the Billing through the UI

b. Admin can delete the Billing

Method 1: Delete the Billing by Name

Method 2: Delete the Billing by category

Method 3: Delete the Billing by Id

c. Admin can Edit the Billing

Method 1: Edit the Billing by Name

Method 2: Edit the Billing by category

Method 3: Edit the Billing by Id

d. Admin can view the Billing

Method 1: view the Billing by Name

Method 2: view the Billing by category

Method 3: View the Billing by Id

6. Tracking Customers module

a. Admin can add tracking customer details

Method 1: Add the tracking customer details directly in the MySQL table using MVC through API

Method 2: Add the tracking customer details through the Excel

Method 3: Add the tracking customer details through the UI

b. Admin can delete the tracking customer details

Method 1: Delete the tracking customer details by Name

Method 2: Delete the tracking customer details by category

Method 3: Delete the tracking customer details by Id

c. Admin can Edit the tracking customer details

Method 1: Edit the tracking customer details by Name

Method 2: Edit the tracking customer details by category

Method 3: Edit the tracking customer details by Id

d. Admin can view the tracking customer details

Method 1: view the tracking customer details by Name

Method 2: view the tracking customer details by category

Method 3: view the tracking customer details by Id

7. Tracking Customers module

a. Admin can add tracking customer details

Method 1: Add the tracking customer details directly in the MySQL table using MVC through API

Method 2: Add the tracking customer details through the Excel

Method 3: Add the tracking customer details through the UI

b. Admin can delete the tracking customer details

Method 1: Delete the tracking customer details by Name

Method 2: Delete the tracking customer details by category

Method 3: Delete the tracking customer details by Id

c. Admin can Edit the tracking customer details

Method 1: Edit the tracking customer details by Name

Method 2: Edit the tracking customer details by category

Method 3: Edit the tracking customer details by Id

d. Admin can view the tracking customer details

Method 1: view the tracking customer details by Name

Method 2: view the tracking customer details by category

Method 3: view the tracking customer details by Id

8. Inventory Tracking

a. Admin can add Inventory Items, Quantities

Method 1: Add the Inventory Items, Quantities directly in the MySQL table using MVC through API

Method 2: Add the Inventory Items, Quantities through the Excel

Method 3: Add the Inventory Items, Quantities through the UI

b. Admin can delete the Inventory Items, Quantities

Method 1: Delete the Inventory Items, Quantities by Name

Method 2: Delete the Inventory Items, Quantities by category

Method 3: Delete the Inventory Items, Quantities by Id

c. Admin can Edit the Inventory Items, Quantities

Method 1: Edit the Inventory Items, Quantities by Name

Method 2: Edit the Inventory Items, Quantities by category

Method 3: Edit the Inventory Items, Quantities by Id

d. Admin can view the Inventory Items, Quantities

Method 1: view the Inventory Items, Quantities by Name

Method 2: view the Inventory Items, Quantities by category

Method 3: view the Inventory Items, Quantities by Id

Why?	Why not?
1.Admin a. Admin can Login Method 2: Login through Email id and password Login using email id and password is more secure because the email is more unique so no one can duplicate it. For this platform the email id and password factor authentication is more sufficient. This is more secure, and you can login with only the owner of the email. b. Admin can add the Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking Method 1: Add the Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking directly in the MySQL table using MVC through API MySQL is a good practice and easy way to add the more no of Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking in one single table . This is also easy and more comfortable one as well as Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking friendly and easy understanding. This is more sufficient for this platform. c. Admin can delete the Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking Method 3: Delete the Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking by Id Id is the unique one and act as a primary key in the database, so it is more useful one for the deleting the Datas whose having this ID this the right way of deleting the Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking by their id. d. Admin can Edit the Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking Method 3: Edit the Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking by Id	1.Admin a. Admin can Login The Email id and password is more sufficient for this platform and others do have some duplications of names and social profiles so that's not good appropriate one for the secure platform. b. Admin can add the Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking MYSQL is More sufficient when compared to others and the others may leads to change in main table of the database and somewhat difficult to add the Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Trackings in a Single manner. These are the major reasons why I'm not using the other methods. c. Admin can delete the Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking Id is the unique one and act as a primary key in the database, so it is more useful one for the deleting the Datas and others having the duplications of the data those are not the right method for delete the Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking. d. Admin can Edit the Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking Id is the unique one and act as a primary key in the database, so it is more useful one for the editing the Datas and others having the duplications of the data those are not the right method for edit the Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking. e. Admin can view Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking Id is the unique one and act as a primary key in the database, so it is more useful one for the viewing the Datas and others having the

Id is the unique one and act as a primary key in the database, so it is more useful one for the editing the Datas whose having this ID this the right way of editing the Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking by their id.

e. Admin can view the Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking

Method 3: view Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking by Id

Id is the unique one and act as a primary key in the database, so it is more useful one for the viewing the Datas whose having this ID this the right way of viewing the Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking by their id.

2.User

a. User can Login

Method 2: Login through Email id and password

Login using email id and password is more secure because the email is more unique so no one can duplicate it. For this platform the email id and password factor authentication is more sufficient. This is more secure, and you can login with only the owner of the email.

b. User can add the Appointment Scheduling ,Online Booking

Method 1: Add the Appointment Scheduling ,Online Booking directly in the MySQL table using MVC through API

MYSQL is a good practice and easy way to add the more no of Appointment Scheduling ,Online Booking in one single table . This is also easy and more comfortable one as well as Appointment Scheduling ,Online Booking friendly and easy understanding. This is more sufficient for this platform.

c. User can delete the Appointment Scheduling ,Online Booking

Method 3: Delete the Appointment Scheduling ,Online Booking by Id

Id is the unique one and act as a primary key in the database, so it is more useful one for

duplications of the data those are not the right method for view the Appointment Scheduling, Online Booking, Billing, tracking customers, staff Members, Inventory Tracking.

2.User

a. User can Login

The Email id and password is more sufficient for this platform and others do have some duplications of names and social profiles so that's not good appropriate one for the secure platform.

b. User can add the Appointment Scheduling ,Online Booking

MYSQL is More sufficient when compared to others and the others may leads to change in main table of the database and somewhat difficult to add the Appointment Scheduling ,Online Bookings in a Single manner. These are the major reasons why I'm not using the other methods.

c. User can delete the Appointment Scheduling ,Online Booking

Id is the unique one and act as a primary key in the database, so it is more useful one for the deleting the Datas and others having the duplications of the data those are not the right method for delete the Appointment Scheduling ,Online Booking.

d. User can Edit the Appointment Scheduling ,Online Booking

Id is the unique one and act as a primary key in the database, so it is more useful one for the editing the Datas and others having the duplications of the data those are not the right method for edit the Appointment Scheduling ,Online Booking.

e. User can view Appointment Scheduling ,Online Booking

Id is the unique one and act as a primary key in the database, so it is more useful one for the viewing the Datas and others having the duplications of the data those are not the right method for view the Appointment Scheduling ,Online Booking.

3.Appointment Scheduling

a. User can add the Service, Staff, Date and Time

MYSQL is More sufficient when compared to others and the others may leads to change in

the deleting the Datas whose having this ID this the right way of deleting the Appointment Scheduling ,Online Booking by their id.

d. User can Edit the Appointment Scheduling ,Online Booking

Method 3: Edit the Appointment Scheduling ,Online Booking by Id

Id is the unique one and act as a primary key in the database, so it is more useful one for the editing the Datas whose having this ID this the right way of editing the Appointment Scheduling ,Online Booking by their id.

e. User can view the Appointment Scheduling ,Online Booking

Method 3: view Appointment Scheduling ,Online Booking by Id

Id is the unique one and act as a primary key in the database, so it is more useful one for the viewing the Datas whose having this ID this the right way of viewing the Appointment Scheduling ,Online Booking by their id.

3. Appointment Scheduling

a. User can add the Service, Staff, Date and Time

Method 1: Add the Service, Staff, Date and Time directly in the MySQL table using MVC through API

MYSQL is a good practice and easy way to add the more no of Service, Staff, Date and Time in one single table . This is also easy and more comfortable one as well as Service, Staff, Date and Time friendly and easy understanding. This is more sufficient for this platform.

b. User can delete the Service, Staff, Date and Time

Method 3: Delete the Service, Staff, Date and Time by Id

Id is the unique one and act as a primary key in the database, so it is more useful one for the deleting the Datas whose having this ID this the right way of deleting the Service, Staff, Date and Time by their id.

c. User can Edit the Service, Staff, Date and Time

Method 3: Edit the Service, Staff, Date and Time by Id

Id is the unique one and act as a primary key in the database, so it is more useful one for the editing the Datas whose having this ID this the right way of editing the Service, Staff,

main table of the database and somewhat difficult to add the Service, Staff, Date and Times in a Single manner. These are the major reasons why I'm not using the other methods.

b. User can delete the Service, Staff, Date and Time

Id is the unique one and act as a primary key in the database, so it is more useful one for the deleting the Datas and others having the duplications of the data those are not the right method for delete the Service, Staff, Date and Time.

c. User can Edit the Service, Staff, Date and Time

Id is the unique one and act as a primary key in the database, so it is more useful one for the editing the Datas and others having the duplications of the data those are not the right method for edit the Service, Staff, Date and Time.

d. User can view Service, Staff, Date and Time

Id is the unique one and act as a primary key in the database, so it is more useful one for the viewing the Datas and others having the duplications of the data those are not the right method for view the Service, Staff, Date and Time.

4. Online Booking

a. User can add the Online Booking

MYSQL is More sufficient when compared to others and the others may leads to change in main table of the database and somewhat difficult to add the Online Bookings in a Single manner. These are the major reasons why I'm not using the other methods.

b. User can delete the Online Booking

Id is the unique one and act as a primary key in the database, so it is more useful one for the deleting the Datas and others having the duplications of the data those are not the right method for delete the Online Booking .

c. User can Edit the Online Booking

Id is the unique one and act as a primary key in the database, so it is more useful one for the editing the Datas and others having the duplications of the data those are not the right

Date and Time by their id.

d. User can view the Service, Staff, Date and Time

Method 3: view Service, Staff, Date and Time by Id

Id is the unique one and act as a primary key in the database, so it is more useful one for the viewing the Datas whose having this ID this the right way of viewing the Service, Staff, Date and Time by their id.

4. Online Booking

a. User can add the Online Booking

Method 1: Add the Online Booking directly in the MySQL table using MVC through API

MYSQL is a good practice and easy way to add the more no of Online Booking in one single table . This is also easy and more comfortable one as well as Online Booking friendly and easy understanding. This is more sufficient for this platform.

b. User can delete the Online Booking

Method 3: Delete the Online Booking by Id

Id is the unique one and act as a primary key in the database, so it is more useful one for the deleting the Datas whose having this ID this the right way of deleting the Online Booking by their id.

c. User can Edit the Online Booking

Method 3: Edit the Online Booking by Id

Id is the unique one and act as a primary key in the database, so it is more useful one for the editing the Datas whose having this ID this the right way of editing the Online Booking by their id.

d. User can view the Online Booking

Method 3: view Online Booking by Id

Id is the unique one and act as a primary key in the database, so it is more useful one for the viewing the Datas whose having this ID this the right way of viewing the Online Booking by their id.

5. Billing

a. Admin can add the Billing

Method 1: Add the Billing directly in the MySQL table

method for edit the Online Booking .

d. User can view Online Booking

Id is the unique one and act as a primary key in the database, so it is more useful one for the viewing the Datas and others having the duplications of the data those are not the right method for view the Online Booking .

5.Billing

a. Admin can add the Billing

MYSQL is More sufficient when compared to others and the others may leads to change in main table of the database and somewhat difficult to add the Billing s in a Single manner. These are the major reasons why I'm not using the other methods.

b. Admin can delete the Billing

Id is the unique one and act as a primary key in the database, so it is more useful one for the deleting the Datas and others having the duplications of the data those are not the right method for delete the Billing .

c. Admin can Edit the Billing

Id is the unique one and act as a primary key in the database, so it is more useful one for the editing the Datas and others having the duplications of the data those are not the right method for edit the Billing .

d. Admin can view Billing

Id is the unique one and act as a primary key in the database, so it is more useful one for the viewing the Datas and others having the duplications of the data those are not the right method for view the Billing .

6.Tracking Customers

a. Admin can add the tracking customer details

MYSQL is More sufficient when compared to others and the others may leads to change in main table of the database and somewhat difficult to add the tracking customer details s in a Single manner. These are the major reasons why I'm not using the other methods.

b. Admin can delete the tracking customer details

Id is the unique one and act as a primary key in the database, so it is more useful one for the deleting the Datas and others having the duplications of the data those are not the right

using MVC through API

MYSQL is a good practice and easy way to add the more no of Billing in one single table . This is also easy and more comfortable one as well as Billing friendly and easy understanding. This is more sufficient for this platform.

b. Admin can delete the Billing

Method 3: Delete the Billing by Id

Id is the unique one and act as a primary key in the database, so it is more useful one for the deleting the Datas whose having this ID this the right way of deleting the Billing by their id.

c. Admin can Edit the Billing

Method 3: Edit the Billing by Id

Id is the unique one and act as a primary key in the database, so it is more useful one for the editing the Datas whose having this ID this the right way of editing the Billing by their id.

d. Admin can view the Billing

Method 3: view Billing by Id

Id is the unique one and act as a primary key in the database, so it is more useful one for the viewing the Datas whose having this ID this the right way of viewing the Billing by their id.

6. Tracking Customers

a. Admin can add the tracking customer details

Method 1: Add the tracking customer details directly in the MySQL table using MVC through API

MYSQL is a good practice and easy way to add the more no of tracking customer details in one single table . This is also easy and more comfortable one as well as tracking customer details friendly and easy understanding. This is more sufficient for this platform.

b. Admin can delete the tracking customer details

Method 3: Delete the tracking customer details by Id

Id is the unique one and act as a primary key in the database, so it is more useful one for the deleting the Datas whose having this ID this the right way of deleting the tracking

method for delete the tracking customer details .

c. Admin can Edit the tracking customer details

Id is the unique one and act as a primary key in the database, so it is more useful one for the editing the Datas and others having the duplications of the data those are not the right method for edit the tracking customer details .

d. Admin can view tracking customer details

Id is the unique one and act as a primary key in the database, so it is more useful one for the viewing the Datas and others having the duplications of the data those are not the right method for view the tracking customer details .

7. Staff Members

a. Admin can add the Staff Name, Email, Phone, Role, Salon

MYSQL is More sufficient when compared to others and the others may leads to change in main table of the database and somewhat difficult to add the Staff Name, Email, Phone, Role, Salons in a Single manner. These are the major reasons why I'm not using the other methods.

b. Admin can delete the Staff Name, Email, Phone, Role, Salon

Id is the unique one and act as a primary key in the database, so it is more useful one for the deleting the Datas and others having the duplications of the data those are not the right method for delete the Staff Name, Email, Phone, Role, Salon.

c. Admin can Edit the Staff Name, Email, Phone, Role, Salon

Id is the unique one and act as a primary key in the database, so it is more useful one for the editing the Datas and others having the duplications of the data those are not the right method for edit the Staff Name, Email, Phone, Role, Salon.

d. Admin can view Staff Name, Email, Phone, Role, Salon

Id is the unique one and act as a primary key in the database, so it is more useful one for the viewing the Datas and others having the duplications of the data those are not the right method for view Staff Name, Email, Phone, Role, Salon.

customer details by their id.

c. Admin can Edit the tracking customer details

Method 3: Edit the tracking customer details by Id

Id is the unique one and act as a primary key in the database, so it is more useful one for the editing the Datas whose having this ID this the right way of editing the tracking customer details by their id.

d. Admin can view the tracking customer details

Method 3: view tracking customer details by Id

Id is the unique one and act as a primary key in the database, so it is more useful one for the viewing the Datas whose having this ID this the right way of viewing the tracking customer details by their id.

7. Staff Members

a. Admin can add the Staff Name, Email, Phone, Role, Salon

Method 1: Add the Staff Name, Email, Phone, Role, Salon directly in the MySQL table using MVC through API

MYSQL is a good practice and easy way to add the more no of Staff Name, Email, Phone, Role, Salon in one single table. This is also easy and more comfortable one as well as Staff Name, Email, Phone, Role, Salon friendly and easy understanding. This is more sufficient for this platform.

b. Admin can delete the Staff Name, Email, Phone, Role, Salon

Method 3: Delete the Staff Name, Email, Phone, Role, Salon by Id

Id is the unique one and act as a primary key in the database, so it is more useful one for the deleting the Datas whose having this ID this the right way of deleting the Staff Name, Email, Phone, Role, Salon by their id.

c. Admin can Edit the Staff Name, Email, Phone, Role, Salon

Method 3: Edit the Staff Name, Email, Phone, Role, Salon by Id

Id is the unique one and act as a primary key in the database, so it is more useful one for the editing the Datas whose having this ID this the right way of editing the Staff Name, Email, Phone, Role, Salon by their id.

8. Inventory Tracking

a. Admin can add the Inventory Items, Quantities

MYSQL is More sufficient when compared to others and the others may leads to change in main table of the database and somewhat difficult to add the Inventory Items, Quantitiess in a Single manner. These are the major reasons why I'm not using the other methods.

b. Admin can delete the Inventory Items, Quantities

Id is the unique one and act as a primary key in the database, so it is more useful one for the deleting the Datas and others having the duplications of the data those are not the right method for delete the Inventory Items, Quantities.

c. Admin can Edit the Inventory Items, Quantities

Id is the unique one and act as a primary key in the database, so it is more useful one for the editing the Datas and others having the duplications of the data those are not the right method for edit the Inventory Items, Quantities.

d. Admin can view Inventory Items, Quantities

Id is the unique one and act as a primary key in the database, so it is more useful one for the viewing the Datas and others having the duplications of the data those are not the right method for view Inventory Items, Quantities

d. Admin can view the Staff Name, Email, Phone, Role, Salon

Method 3: view Staff Name, Email, Phone, Role, Salon by Id

Id is the unique one and act as a primary key in the database, so it is more useful one for the viewing the Datas whose having this ID this the right way of viewing the Staff Name, Email, Phone, Role, Salon by their id.

8. Inventory Tracking

a. Admin can add the Inventory Items, Quantities

Method 1: Add the Inventory Items, Quantities directly in the MySQL table using MVC through API

MYSQL is a good practice and easy way to add the more no of Inventory Items, Quantities in one single table . This is also easy and more comfortable one as well as Inventory Items, Quantities friendly and easy understanding. This is more sufficient for this platform.

b. Admin can delete the Inventory Items, Quantities

Method 3: Delete the Inventory Items, Quantities by Id

Id is the unique one and act as a primary key in the database, so it is more useful one for the deleting the Datas whose having this ID this the right way of deleting the Inventory Items, Quantities by their id.

c. Admin can Edit the Inventory Items, Quantities

Method 3: Edit the Inventory Items, Quantities by Id

Id is the unique one and act as a primary key in the database, so it is more useful one for the editing the Datas whose having this ID this the right way of editing the Inventory Items, Quantities by their id.

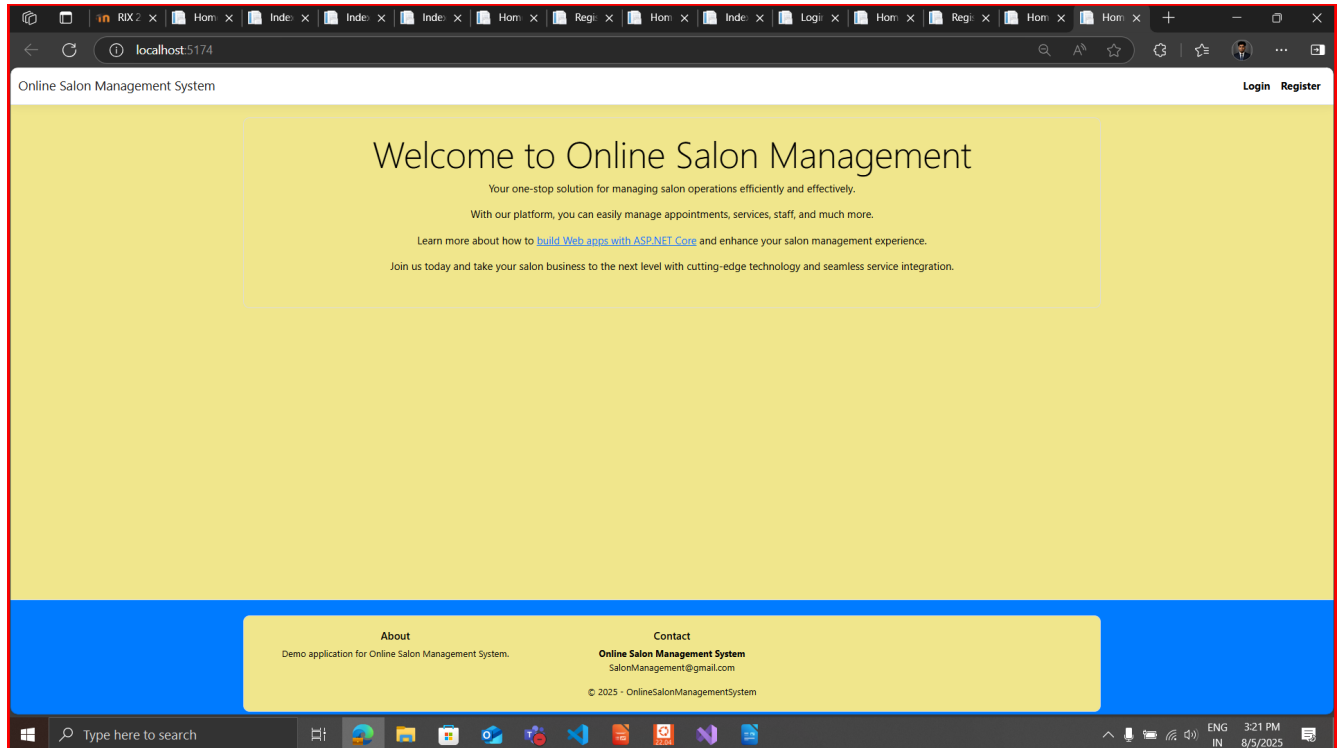
d. Admin can view the Inventory Items, Quantities

Method 3: view Inventory Items, Quantities by Id

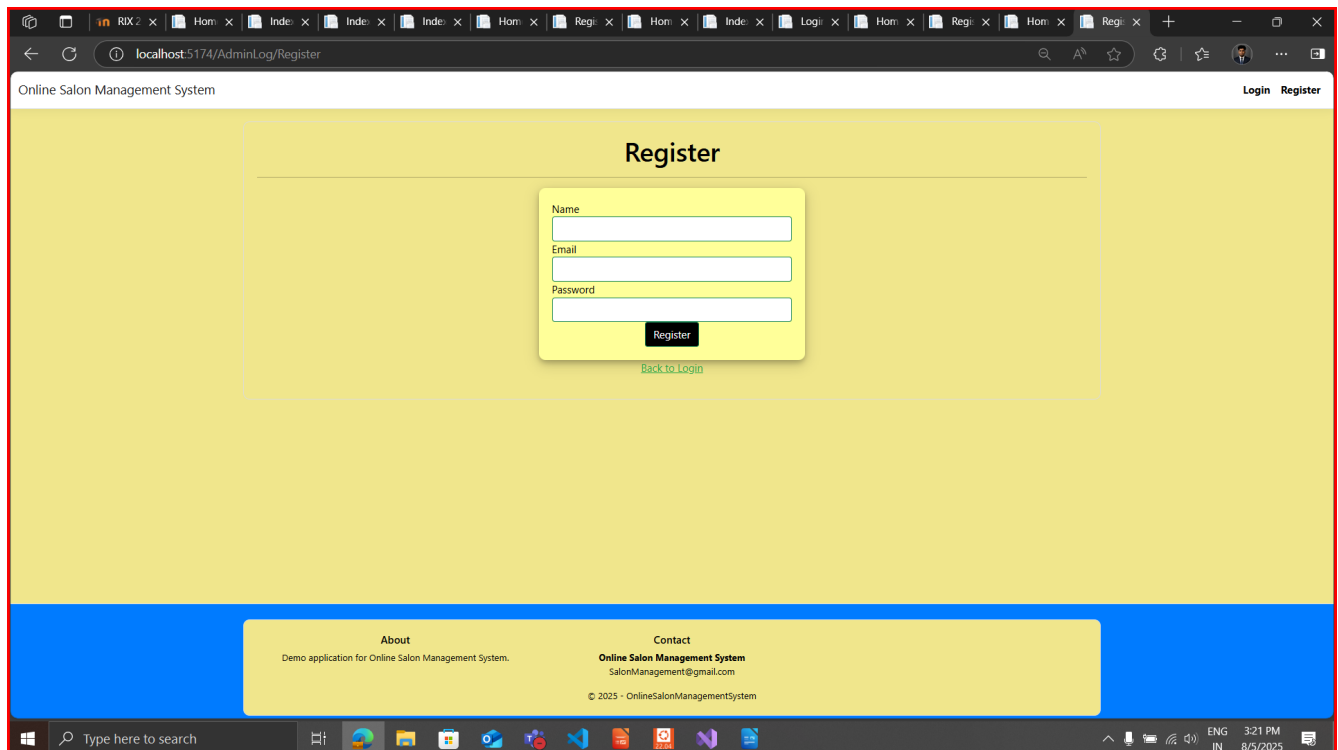
Id is the unique one and act as a primary key in the database, so it is more useful one for the viewing the Datas whose having this ID this the right way of viewing the Inventory Items, Quantities by their id.

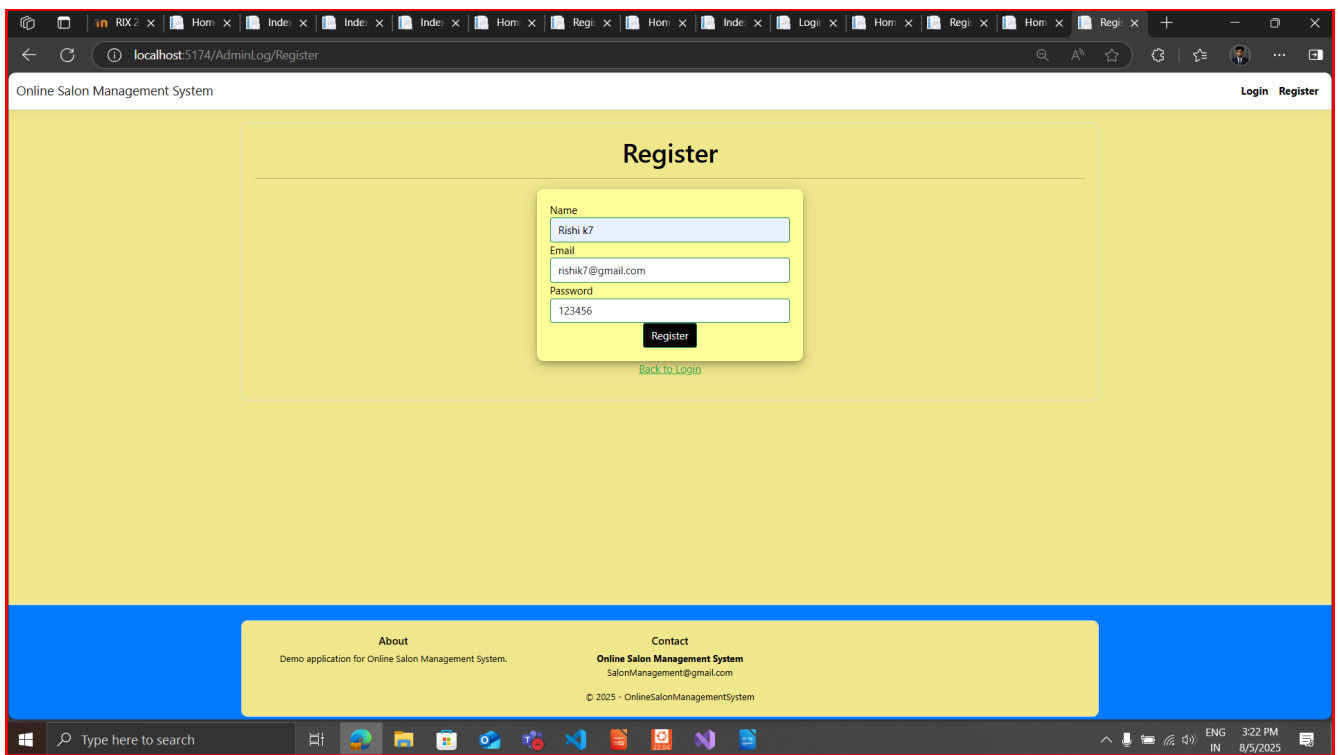
2. User interactive application Screen short doc.

Home Page Before Login/Register

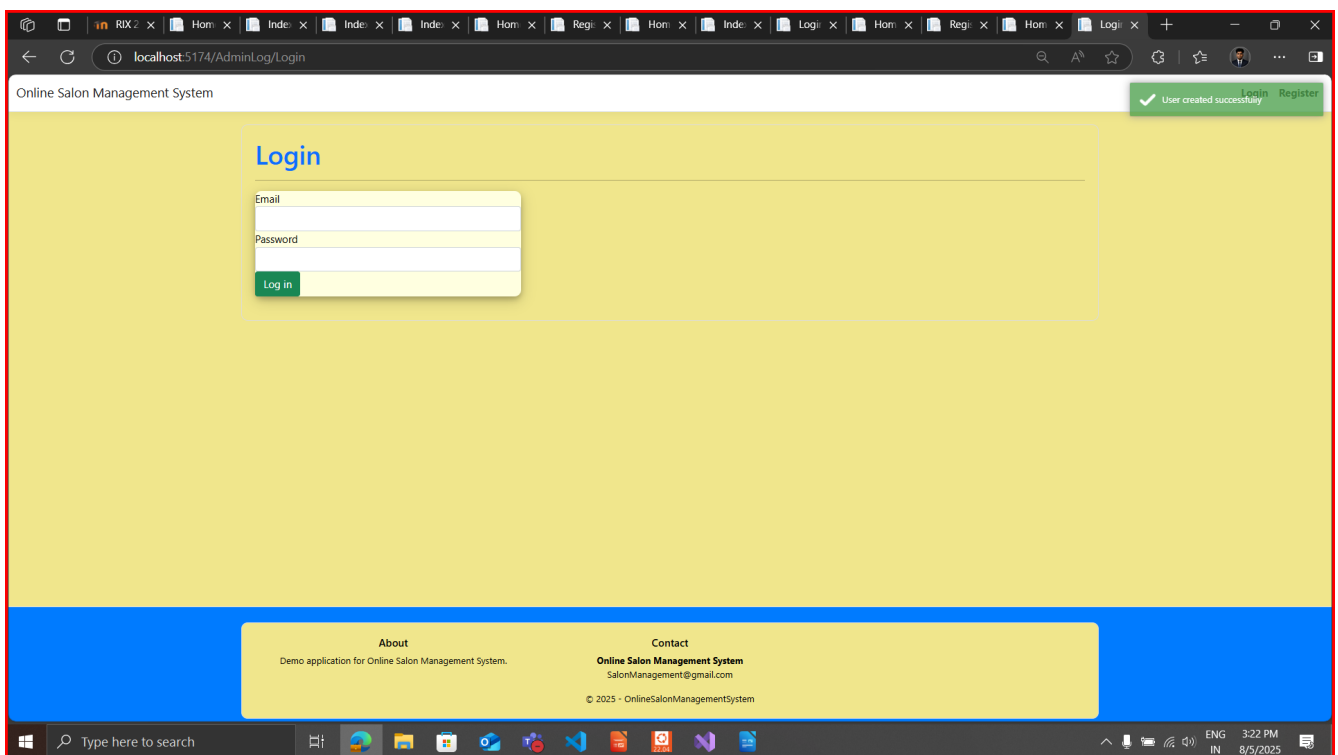


Registration Page

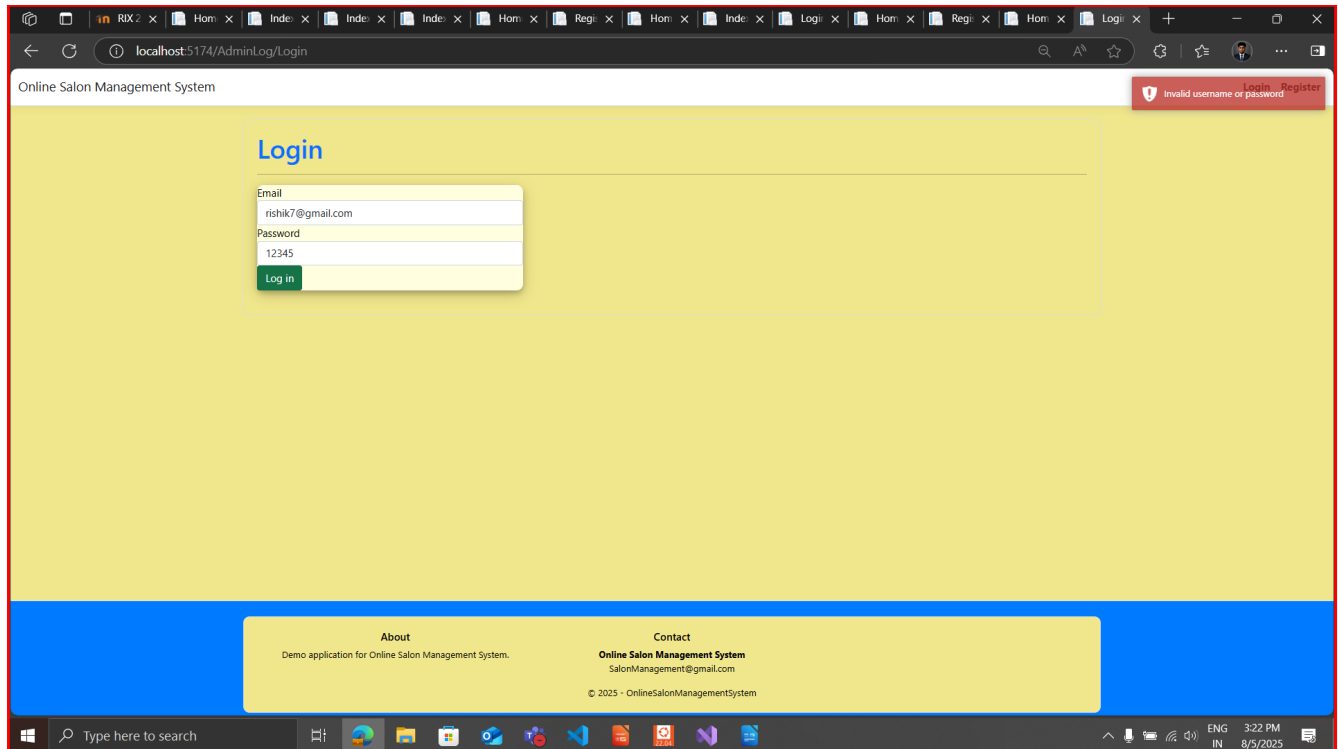




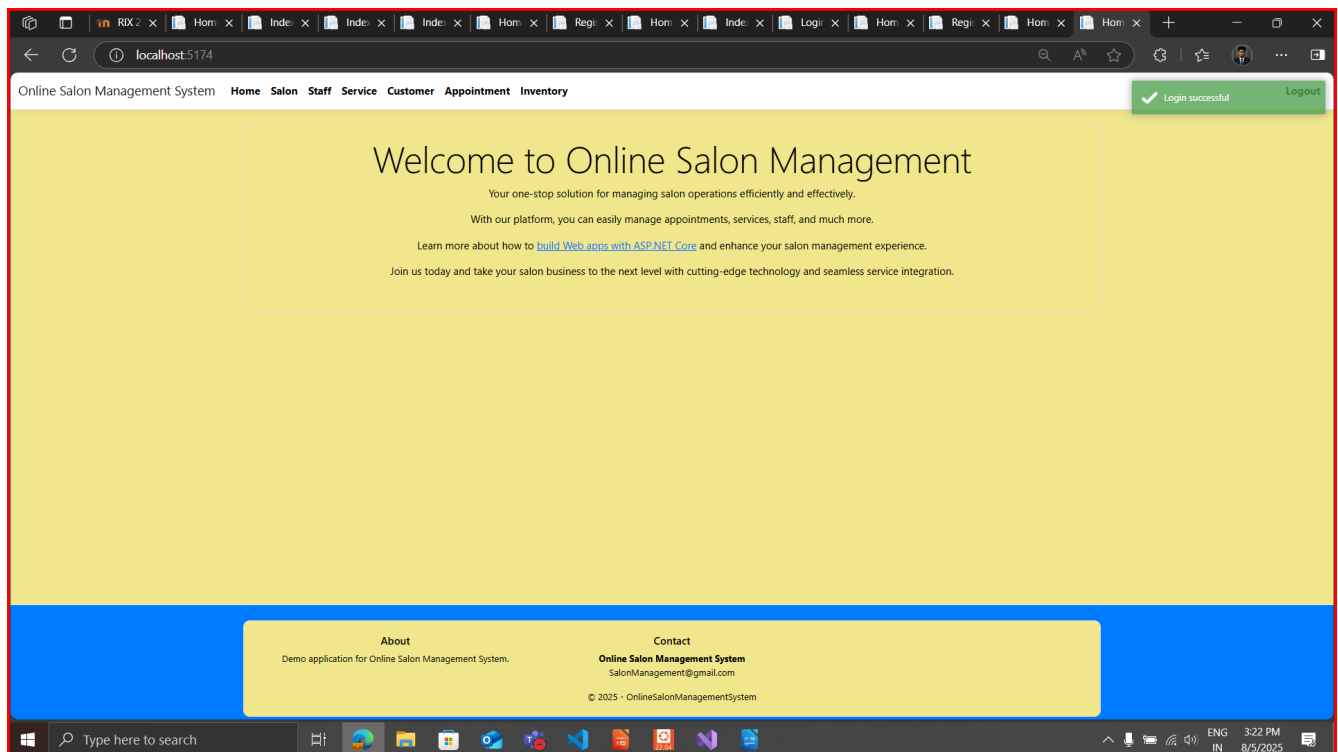
After Register navigate to Login Page and Toaster is Shown



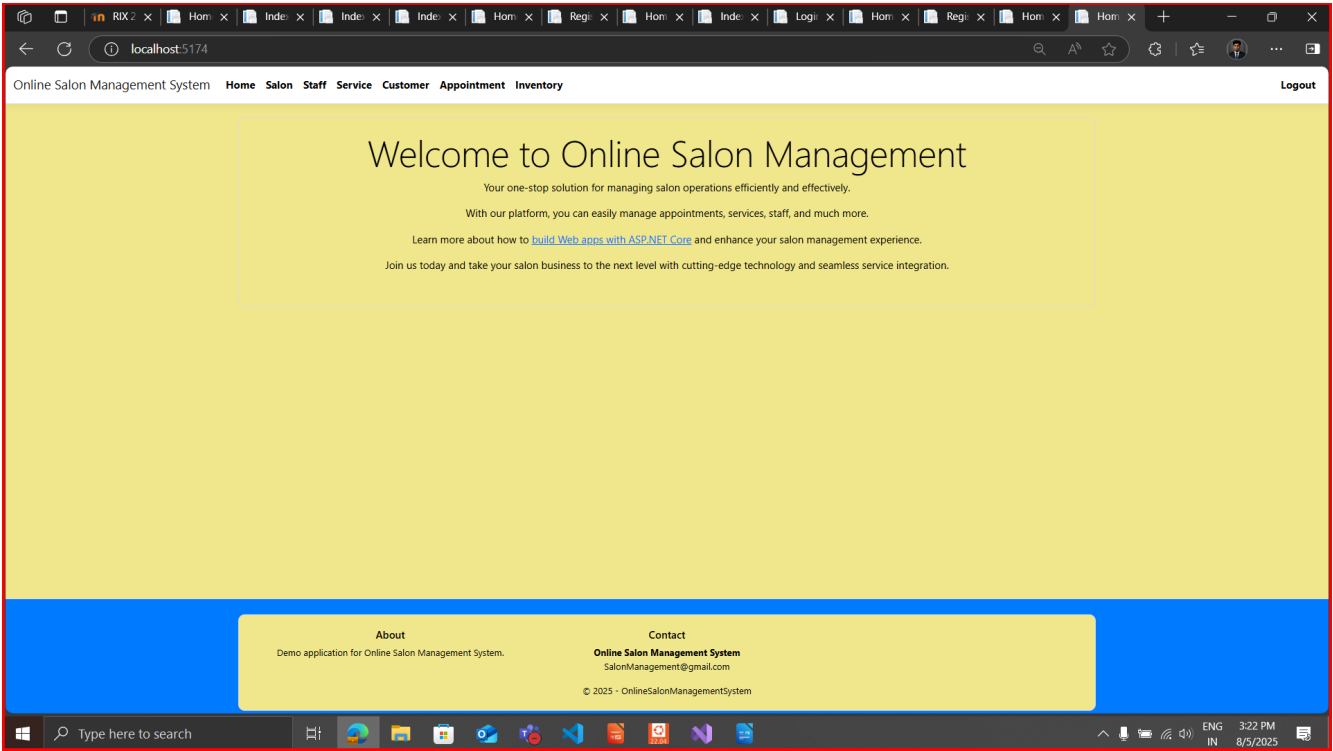
Invalid Credential with Error



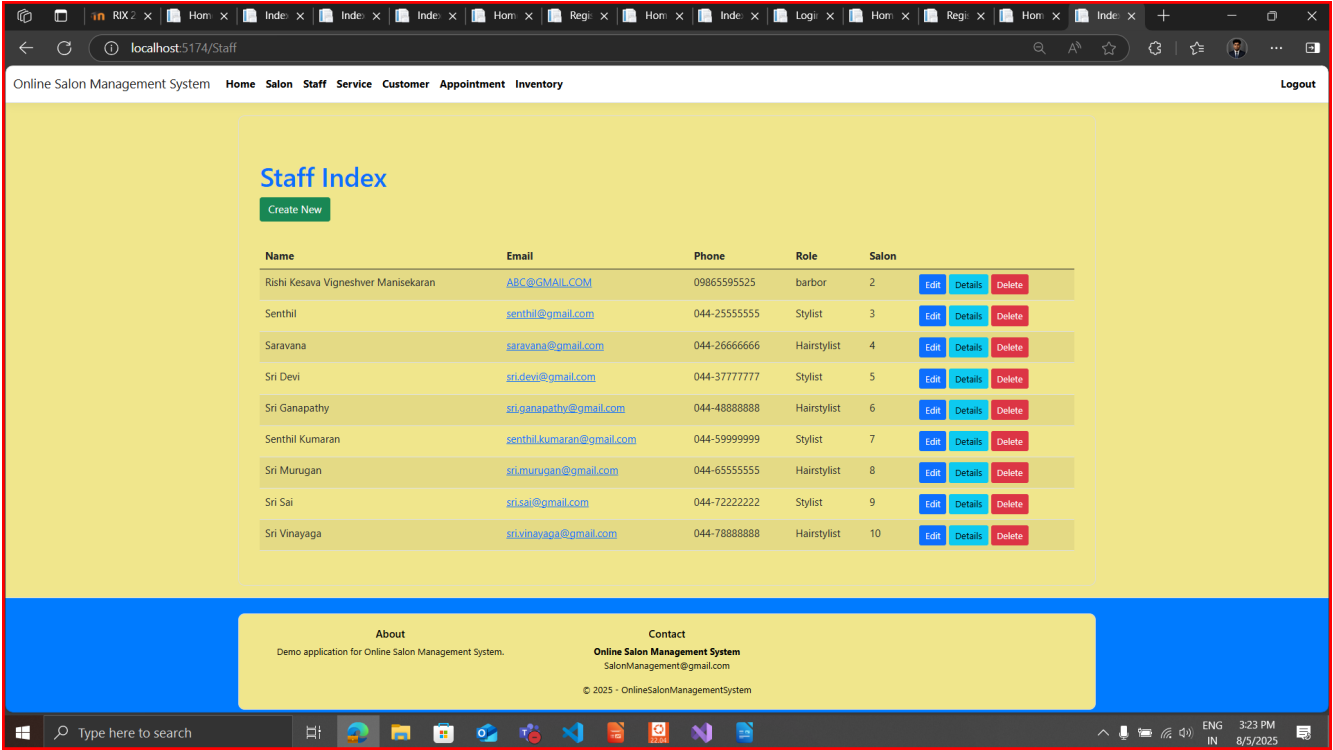
Valid credential and navigate to Home with some Navigation Controllers before login the navigation only having login and register



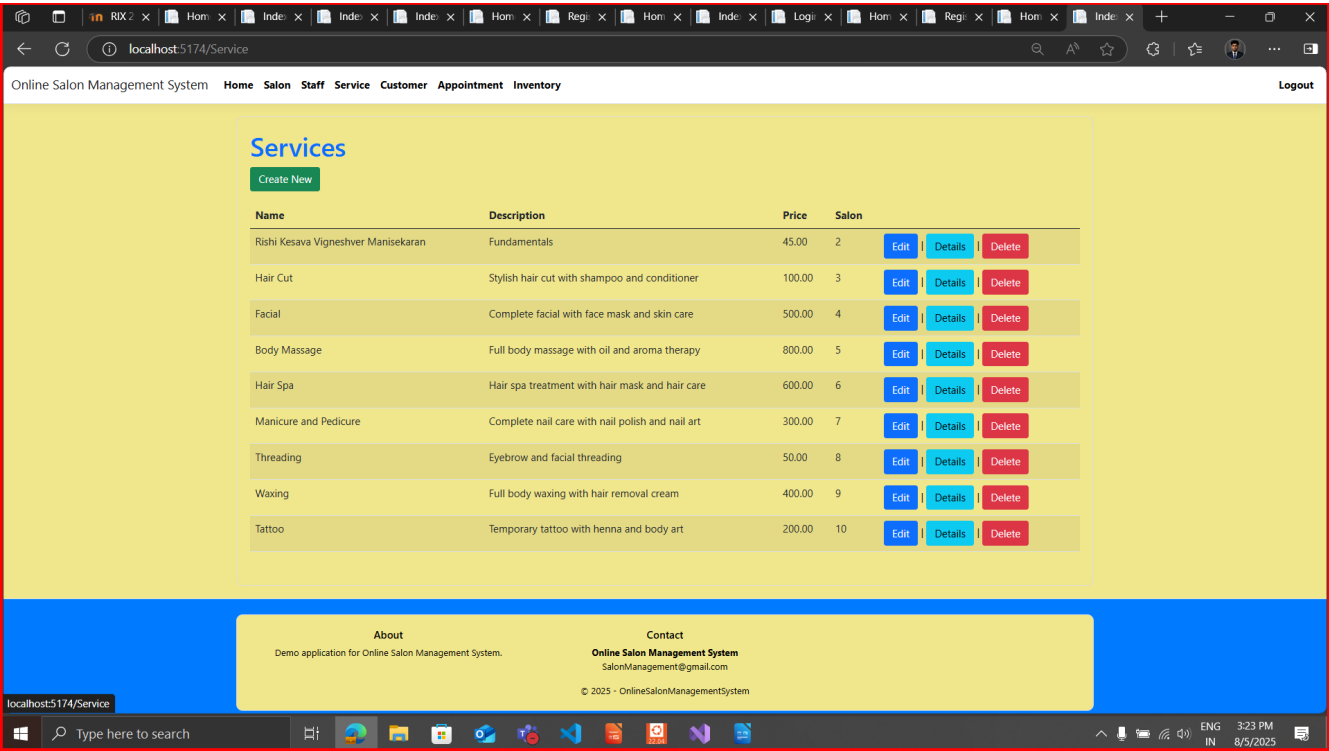
Home Page After Login



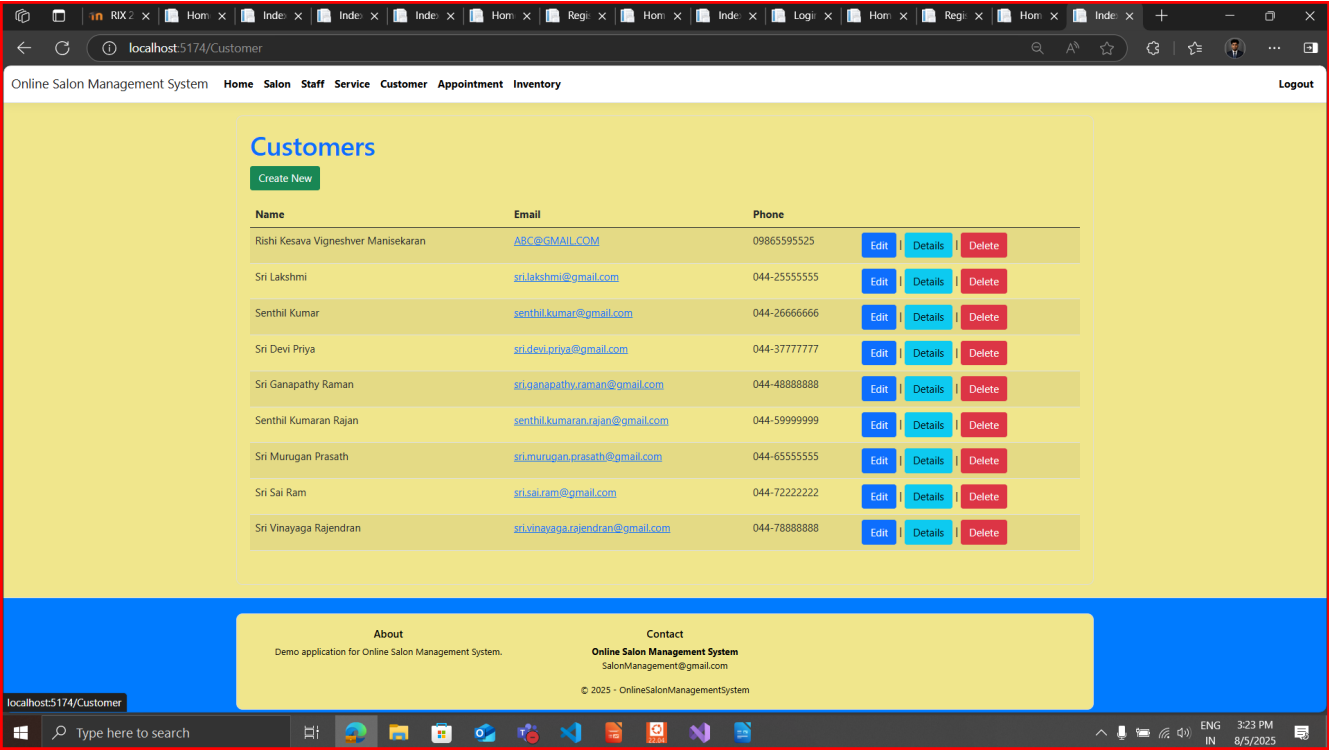
Staff Page



Service Page



Customers Page



Appointment Page

Online Salon Management System

HomeSalonStaffServiceCustomerAppointmentInventory

Logout

Appointments

Create New

Date and Time	Customer	Service	Staff	
21/08/2025 01:06	1	1	1	EditDetailsDelete
01/01/2023 10:00	3	3	3	EditDetailsDelete
02/01/2023 11:00	4	4	4	EditDetailsDelete
03/01/2023 12:00	5	5	5	EditDetailsDelete
04/01/2023 13:00	6	6	6	EditDetailsDelete
05/01/2023 14:00	7	7	7	EditDetailsDelete
06/01/2023 15:00	8	8	8	EditDetailsDelete
07/01/2023 16:00	9	9	9	EditDetailsDelete
07/08/2025 19:57	4	1	1	EditDetailsDelete

About

Demo application for Online Salon Management System.

Contact

Online Salon Management System

SalonManagement@gmail.com

© 2025 - OnlineSalonManagementSystem

localhost:5174/Appointment

Type here to search

ENG IN 3:23 PM 8/5/2025

Inventory Page

Online Salon Management System

HomeSalonStaffServiceCustomerAppointmentInventory

Logout

Inventory Index

Create New

Name	Quantity	Salon	
Royal Entiled	75	Rishi Kesava Vigneshver Manisekaran	EditDetailsDelete
Hair Gel	10	Sri Vinayaga Beauty Parlor	EditDetailsDelete
Hair Oil	20	Senthil Hair Styles	EditDetailsDelete
Face Cream	30	Sri Devi Beauty Saloon	EditDetailsDelete
Shampoo	40	Saravana Hair Dresser	EditDetailsDelete
Conditioner	50	Sri Ganapathy Saloon	EditDetailsDelete
Hair Brush	60	Sri Murugan Hair Styles	EditDetailsDelete
Hair Tie	70	Senthil Kumaran Saloon	EditDetailsDelete
Hair Clip	80	Sri Sai Saloon	EditDetailsDelete

About

Demo application for Online Salon Management System.

Contact

Online Salon Management System

SalonManagement@gmail.com

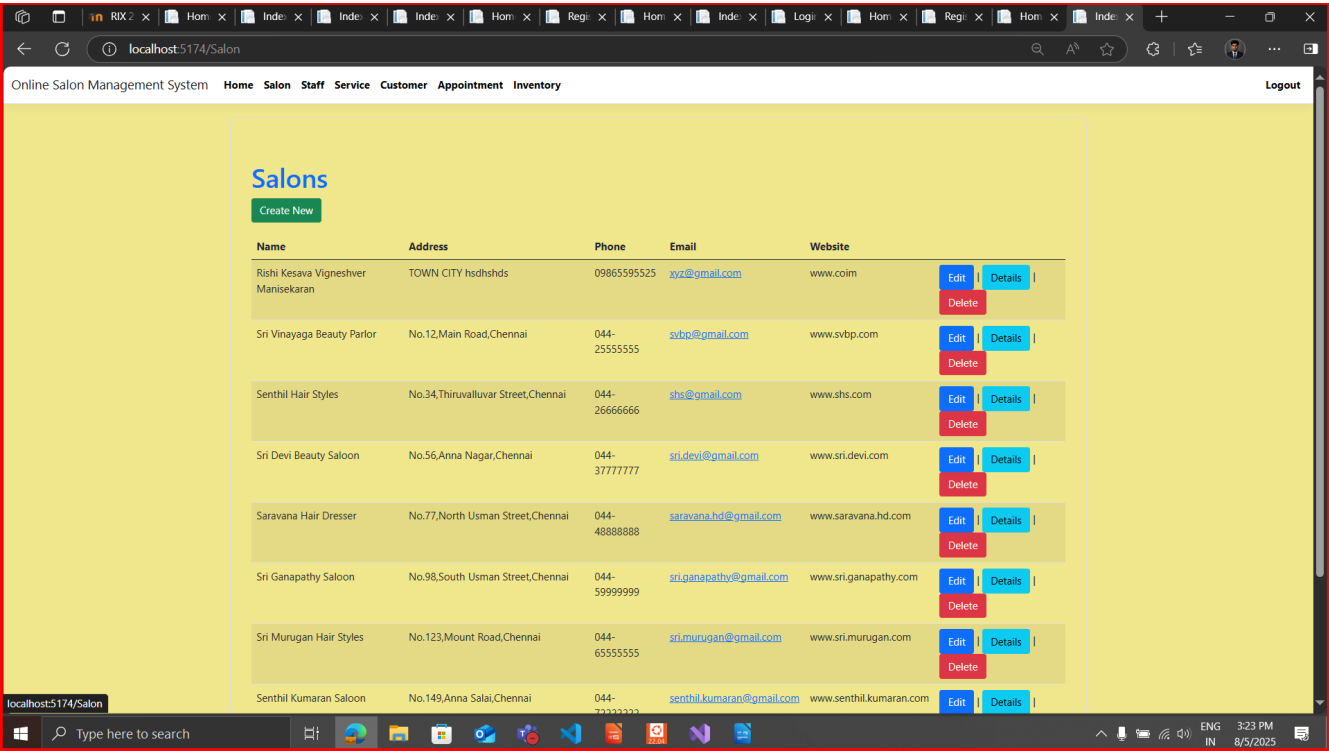
© 2025 - OnlineSalonManagementSystem

localhost:5174/Inventory

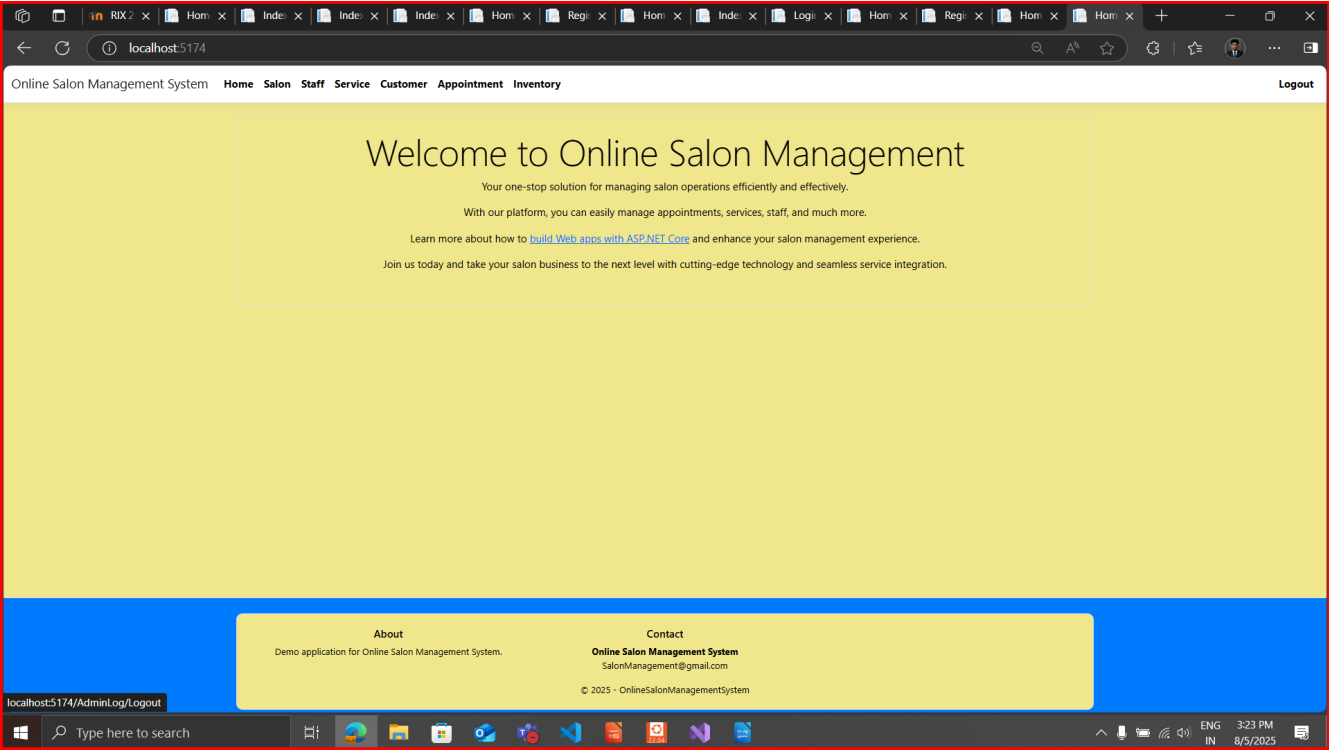
Type here to search

ENG IN 3:23 PM 8/5/2025

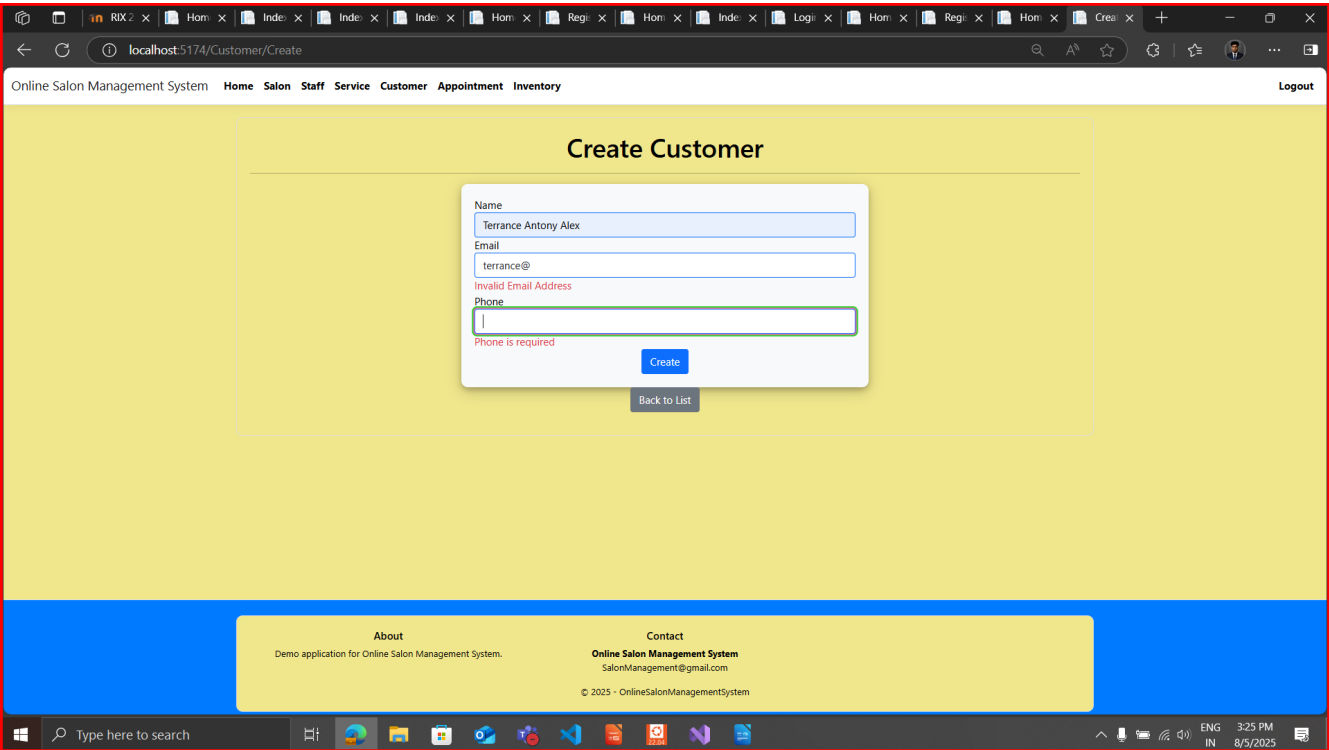
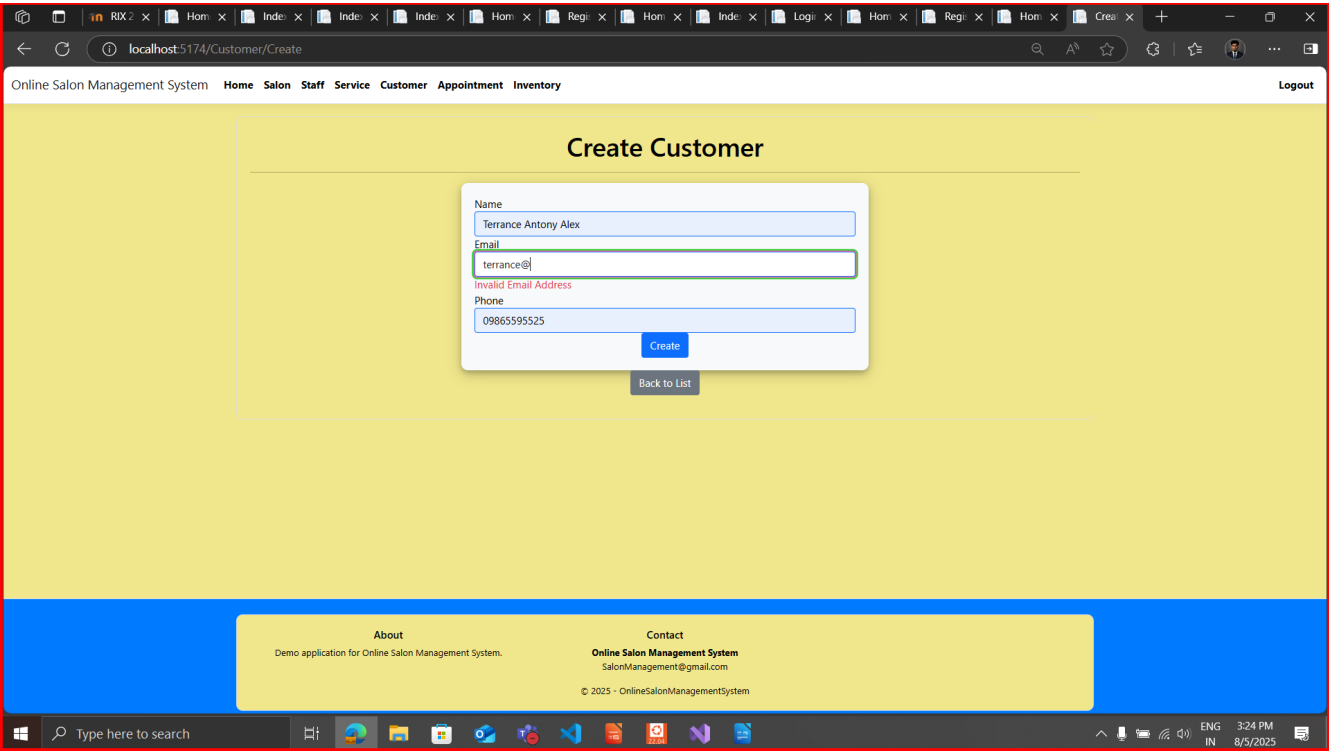
Salon Page



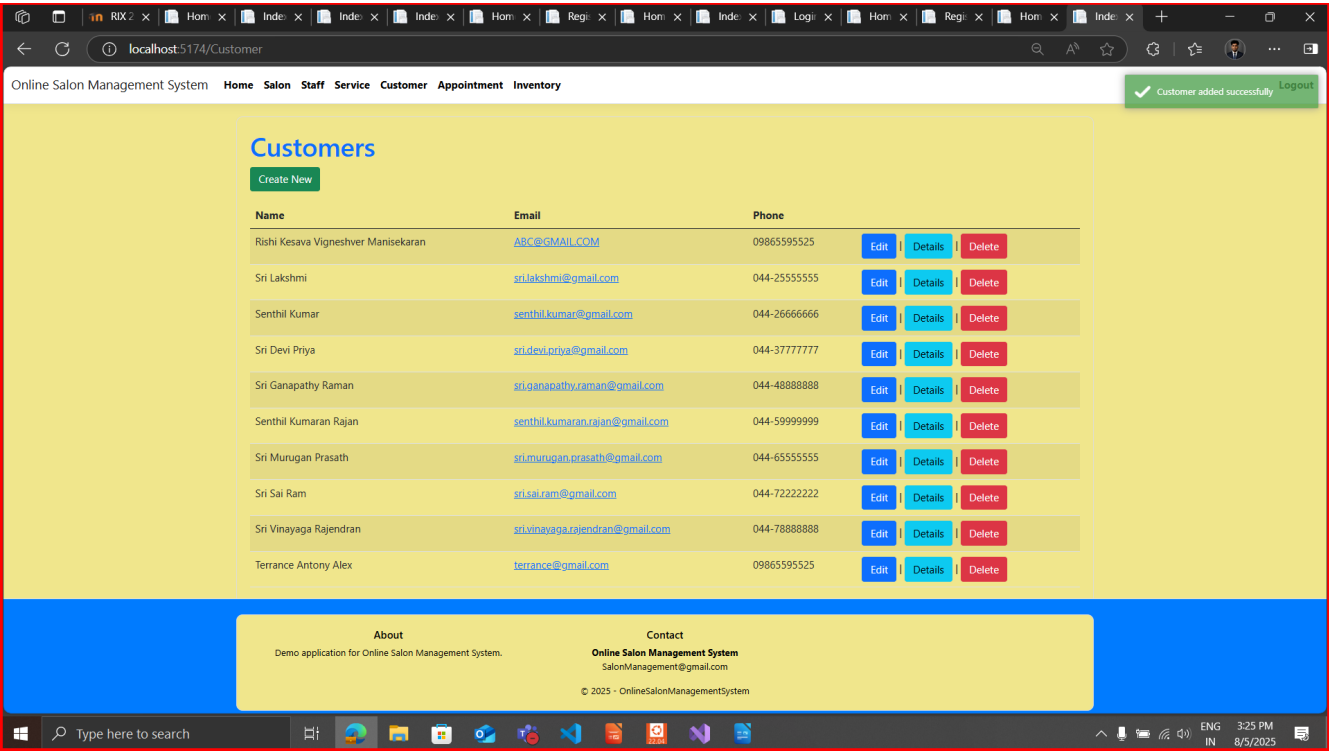
Home



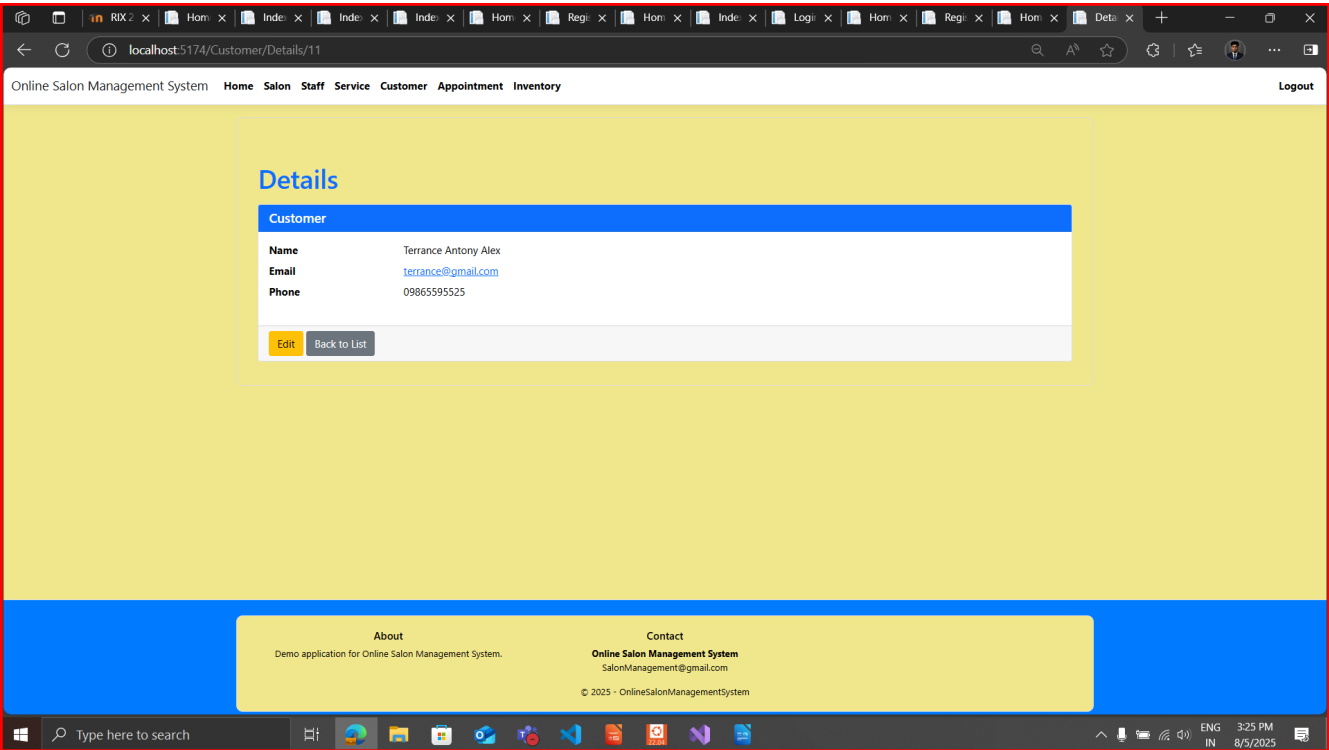
Create Customer with Validation



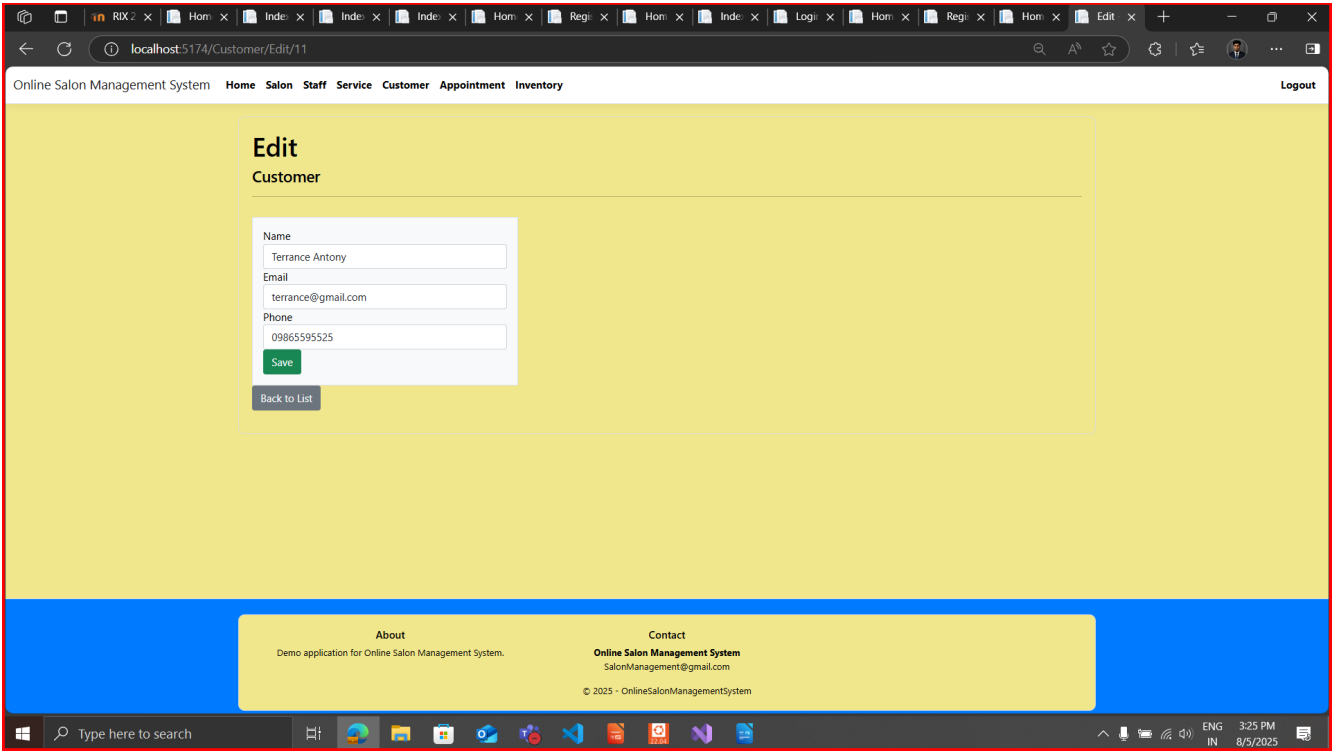
Added Successfully with Toaster Message



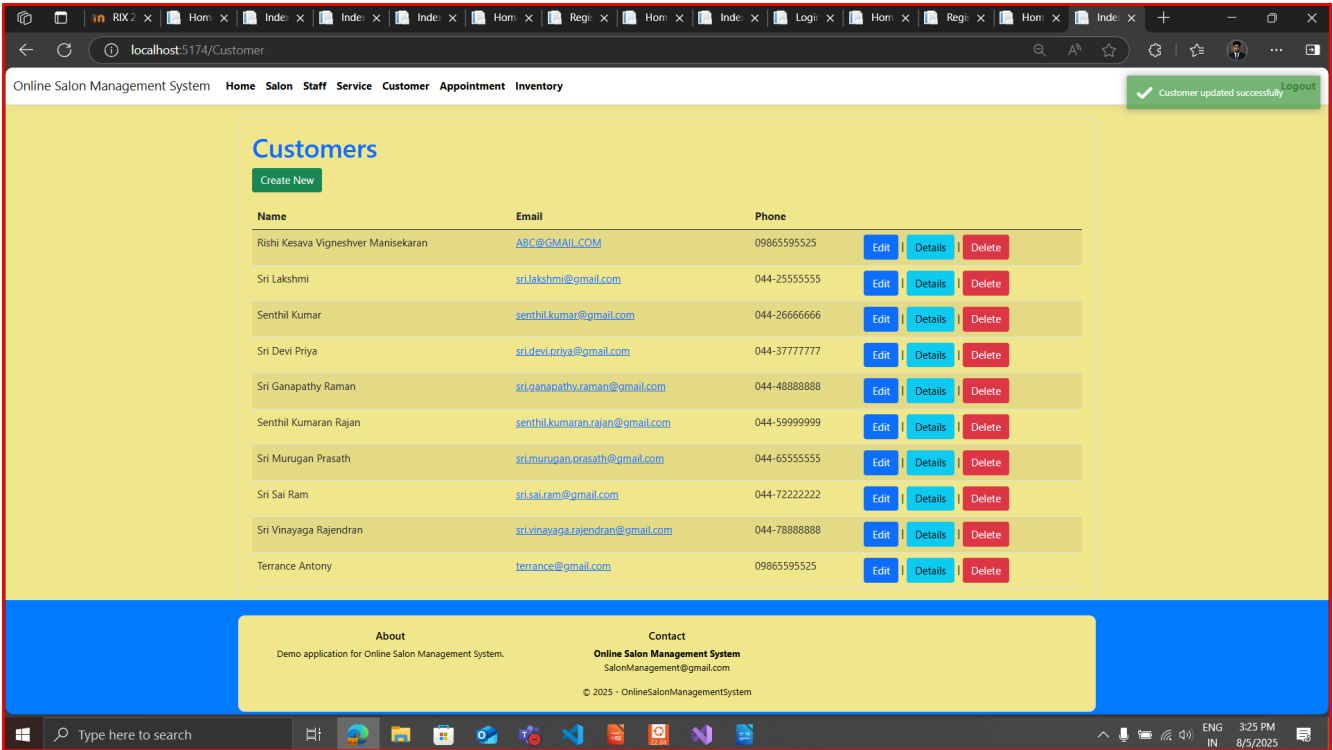
Details



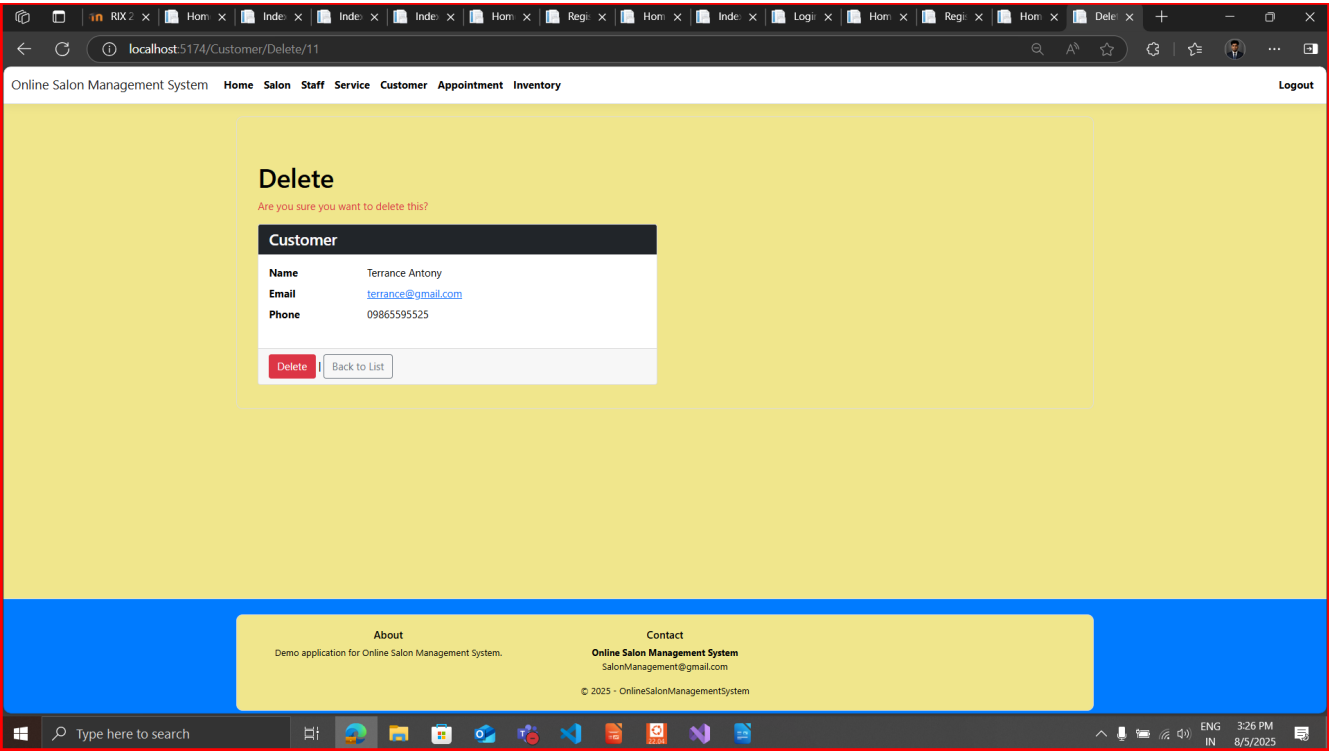
Edit Detail



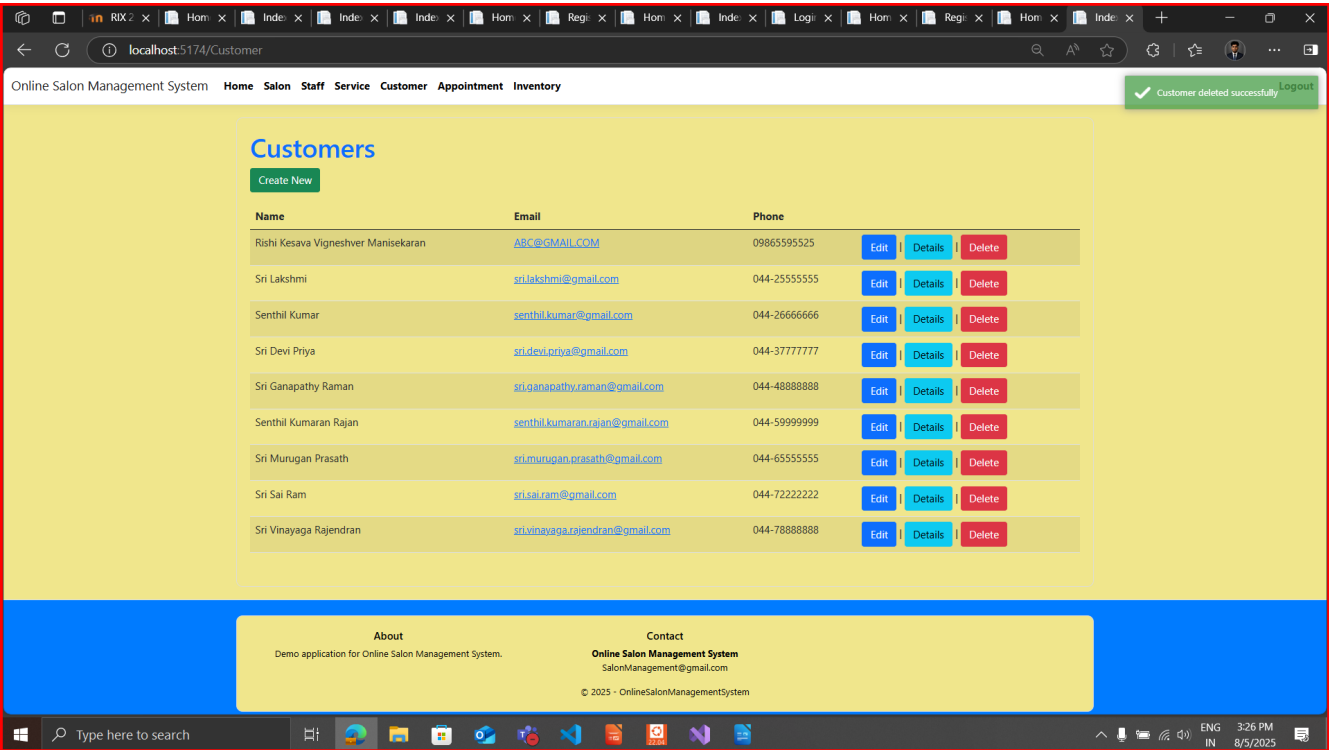
After Edit



Delete

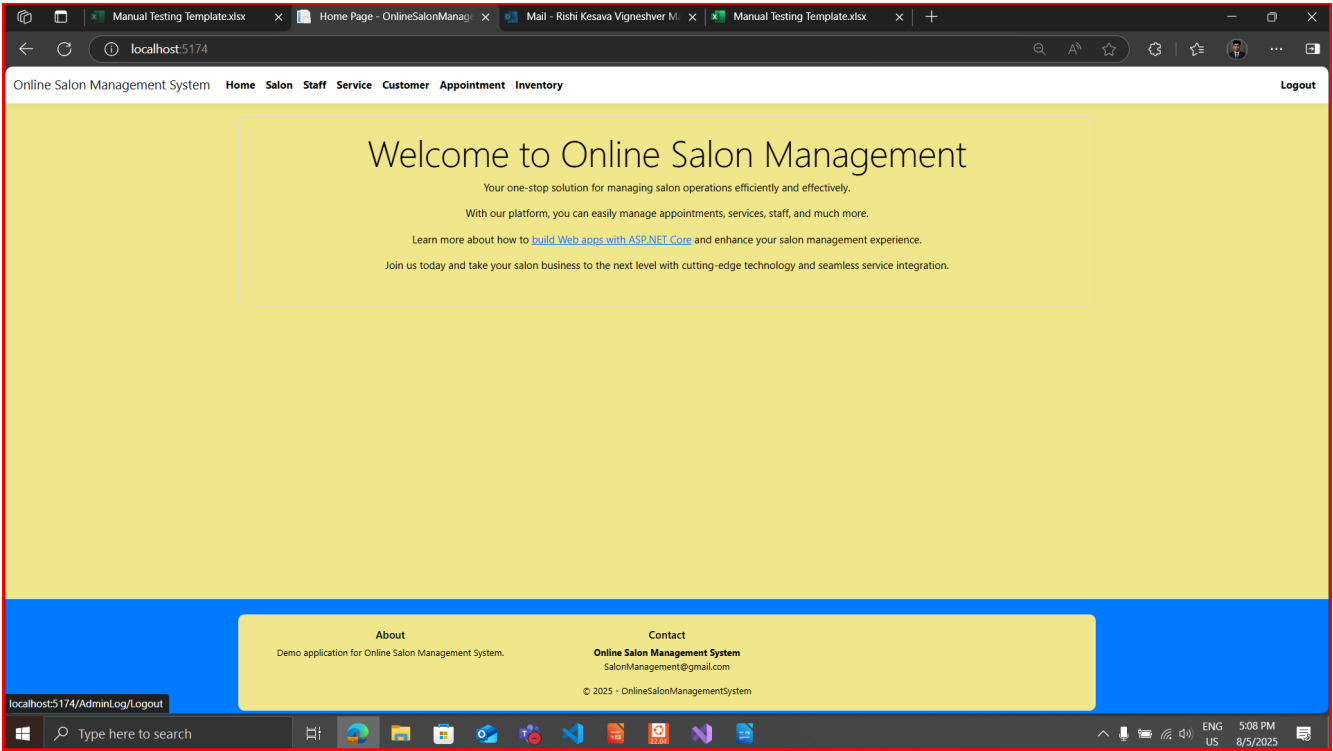


After Delete

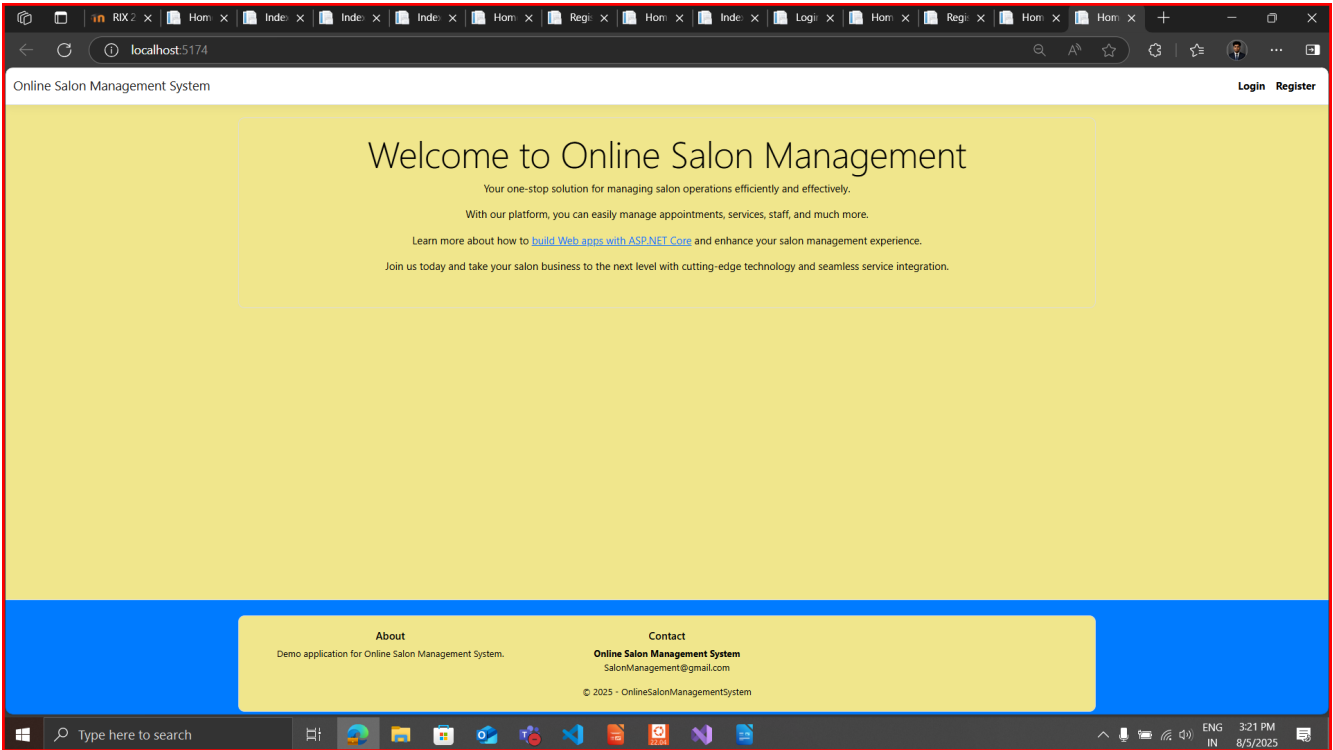


Like the above same operations performed for all other Models

Logout



After Logout it come to normal home page and the controllers will be hide



Codes

Models

AdminLog .cs

```
using System.ComponentModel.DataAnnotations;

namespace OnlineSalonManagementSystem.Models;

public partial class AdminLog
{
    [Key]
    public int Id { get; set; }

    [Required(ErrorMessage = "Name is required")]
    [StringLength(255, ErrorMessage = "Name cannot be longer than 255 characters")]
    public string Name { get; set; } = null!;

    [Required(ErrorMessage = "Email is required")]
    [EmailAddress(ErrorMessage = "Invalid email address")]
    [StringLength(255, ErrorMessage = "Email cannot be longer than 255 characters")]
    public string Email { get; set; } = null!;

    [Required(ErrorMessage = "Password is required")]
    [StringLength(255, ErrorMessage = "Password cannot be longer than 255 characters")]
    public string Password { get; set; } = null!;
}
```

Appointment.cs

```
using System.ComponentModel.DataAnnotations;

namespace OnlineSalonManagementSystem.Models;

public partial class Appointment
{
    [Key]
    [Display(Name = "Appointment ID")]
    public int Id { get; set; }

    [Required]
    [DataType(DataType.DateTime)]
    [Display(Name = "Date and Time")]
    [DisplayFormat(DataFormatString = "{0:dd/MM/yyyy HH:mm}")]
    public DateTime Date { get; set; }

    [Display(Name = "Service ID")]
    public int ServiceId { get; set; }
}
```

```

[Display(Name = "Customer ID")]
public int CustomerId { get; set; }

[Display(Name = "Staff ID")]
public int StaffId { get; set; }

[Display(Name = "Customer")]
public virtual Customer? Customer { get; set; }

[Display(Name = "Service")]
public virtual Service? Service { get; set; }

[Display(Name = "Staff")]
public virtual Staff? Staff { get; set; }
}

```

Customer.cs

```

using System.ComponentModel.DataAnnotations;

namespace OnlineSalonManagementSystem.Models;

public partial class Customer
{
    [Key]
    public int Id { get; set; }

    [Required(ErrorMessage = "Name is required")]
    [StringLength(255, ErrorMessage = "Name cannot be longer than 255 characters")]
    public string Name { get; set; } = null!;

    [Required(ErrorMessage = "Email is required")]
    [StringLength(255, ErrorMessage = "Email cannot be longer than 255 characters")]
    [EmailAddress(ErrorMessage = "Invalid Email Address")]
    public string Email { get; set; } = null!;

    [Required(ErrorMessage = "Phone is required")]
    [StringLength(20, ErrorMessage = "Phone cannot be longer than 20 characters")]
    public string Phone { get; set; } = null!;

    public virtual ICollection<Appointment>? Appointments { get; set; } = new
    List<Appointment>();
}

```

Inventory.cs

```
using System.ComponentModel.DataAnnotations;

namespace OnlineSalonManagementSystem.Models;

public partial class Inventory
{
    [Key]
    public int Id { get; set; }

    [Required(ErrorMessage = "Name is required")]
    [StringLength(255, ErrorMessage = "Name cannot be longer than 255 characters")]
    public string Name { get; set; } = null!;

    [Required(ErrorMessage = "Quantity is required")]
    [Range(0, int.MaxValue, ErrorMessage = "Quantity must be a positive number")]
    public int Quantity { get; set; }

    [Required(ErrorMessage = "Salon is required")]
    public int SalonId { get; set; }

    public virtual Salon? Salon { get; set; } = null!;
}
```

Salon.cs

```
using System.ComponentModel.DataAnnotations;

namespace OnlineSalonManagementSystem.Models;

public partial class Salon
{
    [Key]
    public int Id { get; set; }

    [Required(ErrorMessage = "Name is required")]
    [StringLength(255, ErrorMessage = "Name cannot be longer than 255 characters")]
    public string Name { get; set; } = null!;

    [Required(ErrorMessage = "Address is required")]
    [StringLength(255, ErrorMessage = "Address cannot be longer than 255 characters")]
    public string Address { get; set; } = null!;

    [Required(ErrorMessage = "Phone is required")]
    [StringLength(20, ErrorMessage = "Phone cannot be longer than 20 characters")]
    public string Phone { get; set; } = null!;

    [Required(ErrorMessage = "Email is required")]
    public string Email { get; set; } = null!;
}
```

```

[StringLength(255, ErrorMessage = "Email cannot be longer than 255 characters")]
[EmailAddress(ErrorMessage = "Invalid Email Address")]
public string Email { get; set; } = null!;

[Required(ErrorMessage = "Website is required")]
[StringLength(255, ErrorMessage = "Website cannot be longer than 255 characters")]
public string Website { get; set; } = null!;

public virtual ICollection<Inventory>? Inventories { get; set; } = new
List<Inventory>();

public virtual ICollection<Service>? Services { get; set; } = new List<Service>();

public virtual ICollection<Staff>? Staff { get; set; } = new List<Staff>();
}

```

Service.cs

```

using System;
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;

namespace OnlineSalonManagementSystem.Models;

public partial class Service
{
    public int Id { get; set; }

    [Required(ErrorMessage = "Name is required.")]
    public string Name { get; set; } = null!;

    [Required(ErrorMessage = "Description is required.")]
    public string Description { get; set; } = null!;

    [Range(0.01, double.MaxValue, ErrorMessage = "Price must be greater than 0.")]
    public decimal Price { get; set; }

    public int SalonId { get; set; }

    public virtual ICollection<Appointment>? Appointments { get; set; } = new
List<Appointment>();

    public virtual Salon? Salon { get; set; } = null!;
}

```

Staff.cs

```
using System.ComponentModel.DataAnnotations;

namespace OnlineSalonManagementSystem.Models;

public partial class Staff
{
    [Key]
    public int Id { get; set; }

    [Required(ErrorMessage = "Name is required")]
    [StringLength(255, ErrorMessage = "Name cannot be longer than 255 characters")]
    public string Name { get; set; } = null!;

    [Required(ErrorMessage = "Email is required")]
    [StringLength(255, ErrorMessage = "Email cannot be longer than 255 characters")]
    [EmailAddress(ErrorMessage = "Invalid email address")]
    public string Email { get; set; } = null!;

    [Required(ErrorMessage = "Phone is required")]
    [StringLength(20, ErrorMessage = "Phone cannot be longer than 20 characters")]
    [Phone(ErrorMessage = "Invalid phone number")]
    public string Phone { get; set; } = null!;

    [Required(ErrorMessage = "Role is required")]
    [StringLength(255, ErrorMessage = "Role cannot be longer than 255 characters")]
    public string Role { get; set; } = null!;

    public int SalonId { get; set; }

    public virtual ICollection<Appointment>? Appointments { get; set; } = new
List<Appointment>();

    public virtual Salon? Salon { get; set; } = null!;
}
```


AppDbContext

```
using System;
using System.Collections.Generic;
using Microsoft.EntityFrameworkCore;
using Pomelo.EntityFrameworkCore.MySql.Scaffolding.Internal;

namespace OnlineSalonManagementSystem.Models;

public partial class OnlineSalonManagementDbContext : DbContext
{
    public OnlineSalonManagementDbContext()
    {
    }

    public
OnlineSalonManagementDbContext(DbContextOptions<OnlineSalonManagementDbContext>
options)
        : base(options)
    {
    }

    public virtual DbSet<AdminLog> AdminLogs { get; set; }

    public virtual DbSet<Appointment> Appointments { get; set; }

    public virtual DbSet<Customer> Customers { get; set; }

    public virtual DbSet<Inventory> Inventories { get; set; }

    public virtual DbSet<Salon> Salons { get; set; }

    public virtual DbSet<Service> Services { get; set; }

    public virtual DbSet<Staff> Staffs { get; set; }

    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
#warning To protect potentially sensitive information in your connection string, you
should move it out of source code. You can avoid scaffolding the connection string by
using the Name= syntax to read it from configuration - see
https://go.microsoft.com/fwlink/?linkid=2131148. For more guidance on storing
connection strings, see https://go.microsoft.com/fwlink/?LinkId=723263.
        =>
optionsBuilder.UseMySQL("server=127.0.0.1;user=root;password=123;database=OnlineSalonM
anagementDB;port=3306", Microsoft.EntityFrameworkCore.ServerVersion.Parse("8.0.42-
mysql"));

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder
            .UseCollation("utf8mb4_0900_ai_ci")
```

```

        .HasCharSet("utf8mb4");

modelBuilder.Entity<AdminLog>(entity =>
{
    entity.HasKey(e => e.Id).HasName("PRIMARY");

    entity.ToTable("AdminLog");

    entity.Property(e => e.Email).HasMaxLength(255);
    entity.Property(e => e.Name).HasMaxLength(255);
    entity.Property(e => e.Password).HasMaxLength(255);
});

modelBuilder.Entity<Appointment>(entity =>
{
    entity.HasKey(e => e.Id).HasName("PRIMARY");

    entity.HasIndex(e => e.CustomerId, "CustomerId");

    entity.HasIndex(e => e.ServiceId, "ServiceId");

    entity.HasIndex(e => e.StaffId, "StaffId");

    entity.Property(e => e.Date).HasColumnType("datetime");

    entity.HasOne(d => d.Customer).WithMany(p => p.Appointments)
        .HasForeignKey(d => d.CustomerId)
        .OnDelete(DeleteBehavior.ClientSetNull)
        .HasConstraintName("Appointments_ibfk_2");

    entity.HasOne(d => d.Service).WithMany(p => p.Appointments)
        .HasForeignKey(d => d.ServiceId)
        .OnDelete(DeleteBehavior.ClientSetNull)
        .HasConstraintName("Appointments_ibfk_1");

    entity.HasOne(d => d.Staff).WithMany(p => p.Appointments)
        .HasForeignKey(d => d.StaffId)
        .OnDelete(DeleteBehavior.ClientSetNull)
        .HasConstraintName("Appointments_ibfk_3");
});

modelBuilder.Entity<Customer>(entity =>
{
    entity.HasKey(e => e.Id).HasName("PRIMARY");

    entity.Property(e => e.Email).HasMaxLength(255);
    entity.Property(e => e.Name).HasMaxLength(255);
    entity.Property(e => e.Phone).HasMaxLength(20);
});

modelBuilder.Entity<Inventory>(entity =>
{

```

```

        entity.HasKey(e => e.Id).HasName("PRIMARY");

        entity.HasIndex(e => e.SalonId, "SalonId");

        entity.Property(e => e.Name).HasMaxLength(255);

        entity.HasOne(d => d.Salon).WithMany(p => p.Inventories)
            .HasForeignKey(d => d.SalonId)
            .OnDelete(DeleteBehavior.ClientSetNull)
            .HasConstraintName("Inventories_ibfk_1");
    });

    modelBuilder.Entity<Salon>(entity =>
    {
        entity.HasKey(e => e.Id).HasName("PRIMARY");

        entity.Property(e => e.Address).HasMaxLength(255);
        entity.Property(e => e.Email).HasMaxLength(255);
        entity.Property(e => e.Name).HasMaxLength(255);
        entity.Property(e => e.Phone).HasMaxLength(20);
        entity.Property(e => e.Website).HasMaxLength(255);
    });

    modelBuilder.Entity<Service>(entity =>
    {
        entity.HasKey(e => e.Id).HasName("PRIMARY");

        entity.HasIndex(e => e.SalonId, "SalonId");

        entity.Property(e => e.Description).HasMaxLength(255);
        entity.Property(e => e.Name).HasMaxLength(255);
        entity.Property(e => e.Price).HasPrecision(10, 2);

        entity.HasOne(d => d.Salon).WithMany(p => p.Services)
            .HasForeignKey(d => d.SalonId)
            .OnDelete(DeleteBehavior.ClientSetNull)
            .HasConstraintName("Services_ibfk_1");
    });

    modelBuilder.Entity<Staff>(entity =>
    {
        entity.HasKey(e => e.Id).HasName("PRIMARY");

        entity.HasIndex(e => e.SalonId, "SalonId");

        entity.Property(e => e.Email).HasMaxLength(255);
        entity.Property(e => e.Name).HasMaxLength(255);
        entity.Property(e => e.Phone).HasMaxLength(20);
        entity.Property(e => e.Role).HasMaxLength(255);

        entity.HasOne(d => d.Salon).WithMany(p => p.Staff)
            .HasForeignKey(d => d.SalonId)

```

```

        .OnDelete(DeleteBehavior.ClientSetNull)
        .HasConstraintName("Staffs_ibfk_1");
    });

    OnModelCreatingPartial(modelBuilder);
}

partial void OnModelCreatingPartial(ModelBuilder modelBuilder);
}

```

Program.cs

```

//pcs
using Microsoft.EntityFrameworkCore;
using OnlineSalonManagementSystem.Models;
using Microsoft.AspNetCore.Authentication;
using Microsoft.AspNetCore.Authentication.Cookies;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

builder.Services.AddDbContext<OnlineSalonManagementDbContext>(options =>
    options.UseMySQL(builder.Configuration.GetConnectionString("DefaultConnection"),
        Microsoft.EntityFrameworkCore.ServerVersion.Parse("8.0.42-mysql")));

builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(options =>
    {
        options.LoginPath = "/User/Login";
        options.LogoutPath = "/User/Logout";
    });

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();

app.UseAuthentication();
app.UseAuthorization();

app.MapControllerRoute(

```

```
name: "default",
pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

MySQL

Tables

```
mysql> show tables;
+-----+
| Tables_in_OnlineSalonManagementDB |
+-----+
| AdminLog |
| Appointments |
| Customers |
| Inventories |
| Salons |
| Services |
| Staffs |
+-----+
7 rows in set (0.04 sec)
```

Data in AdminLog

```
mysql> select * from AdminLog;
+----+-----+-----+-----+
| Id | Name | Email | Password |
+----+-----+-----+-----+
| 1 | Rishi Kesava Vigneshver Manisekaran | ABC@GMAIL.COM | 123 |
| 2 | Royal Enfiled | admin@relevantz.com | 123 |
| 3 | Senthil | senthil@gmail.com | senthil123 |
| 4 | Saravana | saravana@gmail.com | saravana123 |
| 5 | Sri Devi | sri.devi@gmail.com | sri.devi123 |
| 6 | Sri Ganapathy | sri.ganapathy@gmail.com | sri.ganapathy123 |
| 7 | Senthil Kumaran | senthil.kumaran@gmail.com | senthil.kumaran123 |
| 8 | Sri Murugan | sri.murugan@gmail.com | sri.murugan123 |
| 9 | Sri Sai | sri.sai@gmail.com | sri.sai123 |
| 10 | Sri Vinayaga | sri.vinayaga@gmail.com | sri.vinayaga123 |
| 11 | naveen | naveen@GMAIL.COM | 123 |
| 12 | Rishi k7 | rishik7@gmail.com | 123456 |
+----+-----+-----+-----+
12 rows in set (0.00 sec)
```

Data in Appointment

```
mysql> select * from Appointments;
```

Id	Date	ServiceId	CustomerId	StaffId
1	2025-08-21 01:06:00	1	1	1
3	2023-01-01 10:00:00	3	3	3
4	2023-01-02 11:00:00	4	4	4
5	2023-01-03 12:00:00	5	5	5
6	2023-01-04 13:00:00	6	6	6
7	2023-01-05 14:00:00	7	7	7
8	2023-01-06 15:00:00	8	8	8
9	2023-01-07 16:00:00	9	9	9
11	2025-08-07 19:57:00	1	4	1

9 rows in set (0.00 sec)

Data in Customers

```
mysql> select * from Customers;
```

Id	Name	Email	Phone
1	Rishi Kesava Vigneshver Manisekaran	ABC@GMAIL.COM	09865595525
3	Sri Lakshmi	sri.lakshmi@gmail.com	044-25555555
4	Senthil Kumar	senthil.kumar@gmail.com	044-26666666
5	Sri Devi Priya	sri.devi.priya@gmail.com	044-37777777
6	Sri Ganapathy Raman	sri.ganapathy.raman@gmail.com	044-48888888
7	Senthil Kumaran Rajan	senthil.kumaran.rajan@gmail.com	044-59999999
8	Sri Murugan Prasath	sri.murugan.prasath@gmail.com	044-65555555
9	Sri Sai Ram	sri.sai.ram@gmail.com	044-72222222
10	Sri Vinayaga Rajendran	sri.vinayaga.rajendran@gmail.com	044-78888888

9 rows in set (0.00 sec)

Data in Inventories

```
mysql> select * from Inventories;
```

Id	Name	Quantity	SalonId
1	Royal Enfiled	75	2
3	Hair Gel	10	3
4	Hair Oil	20	4
5	Face Cream	30	5
6	Shampoo	40	6
7	Conditioner	50	7
8	Hair Brush	60	8
9	Hair Tie	70	9
10	Hair Clip	80	10

9 rows in set (0.00 sec)

Data in Salons

Id	Name	Address	Phone	Email	Website
2	Rishi Kesava Vigneshver Manisekaran	TOWN CITY hsdhshds	09865595525	xyz@gmail.com	www.coim
3	Sri Vinayaga Beauty Parlor	No.12,Main Road,Chennai	044-25555555	svbp@gmail.com	www.svbp.com
4	Senthil Hair Styles	No.34,Thiruvalluvar Street,Chennai	044-26666666	shs@gmail.com	www.shs.com
5	Sri Devi Beauty Saloon	No.56,Anna Nagar,Chennai	044-37777777	sri.devi@gmail.com	www.sri.devi.com
6	Saravana Hair Dresser	No.77,North Usman Street,Chennai	044-48888888	saravana.hd@gmail.com	www.saravana.hd.com
7	Sri Ganapathy Saloon	No.98,South Usman Street,Chennai	044-59999999	sri.ganapathy@gmail.com	www.sri.ganapathy.com
8	Sri Murugan Hair Styles	No.123,Mount Road,Chennai	044-65555555	sri.murugan@gmail.com	www.sri.murugan.com
9	Senthil Kumaran Saloon	No.149,Anna Salai,Chennai	044-72222222	senthil.kumaran@gmail.com	www.senthil.kumaran.com
10	Sri Sai Saloon	No.168,Chennai-Bangalore Highway,Chennai	044-78888888	sri.sai@gmail.com	www.sri.sai.com

9 rows in set (0.00 sec)

Data in Services

```
mysql> select * from Services;
```

Id	Name	Description	Price	SalonId
1	Rishi Kesava Vigneshver Manisekaran	Fundamentals	45.00	2
3	Hair Cut	Stylish hair cut with shampoo and conditioner	100.00	3
4	Facial	Complete facial with face mask and skin care	500.00	4
5	Body Massage	Full body massage with oil and aroma therapy	800.00	5
6	Hair Spa	Hair spa treatment with hair mask and hair care	600.00	6
7	Manicure and Pedicure	Complete nail care with nail polish and nail art	300.00	7
8	Threading	Eyebrow and facial threading	50.00	8
9	Waxing	Full body waxing with hair removal cream	400.00	9
10	Tattoo	Temporary tattoo with henna and body art	200.00	10

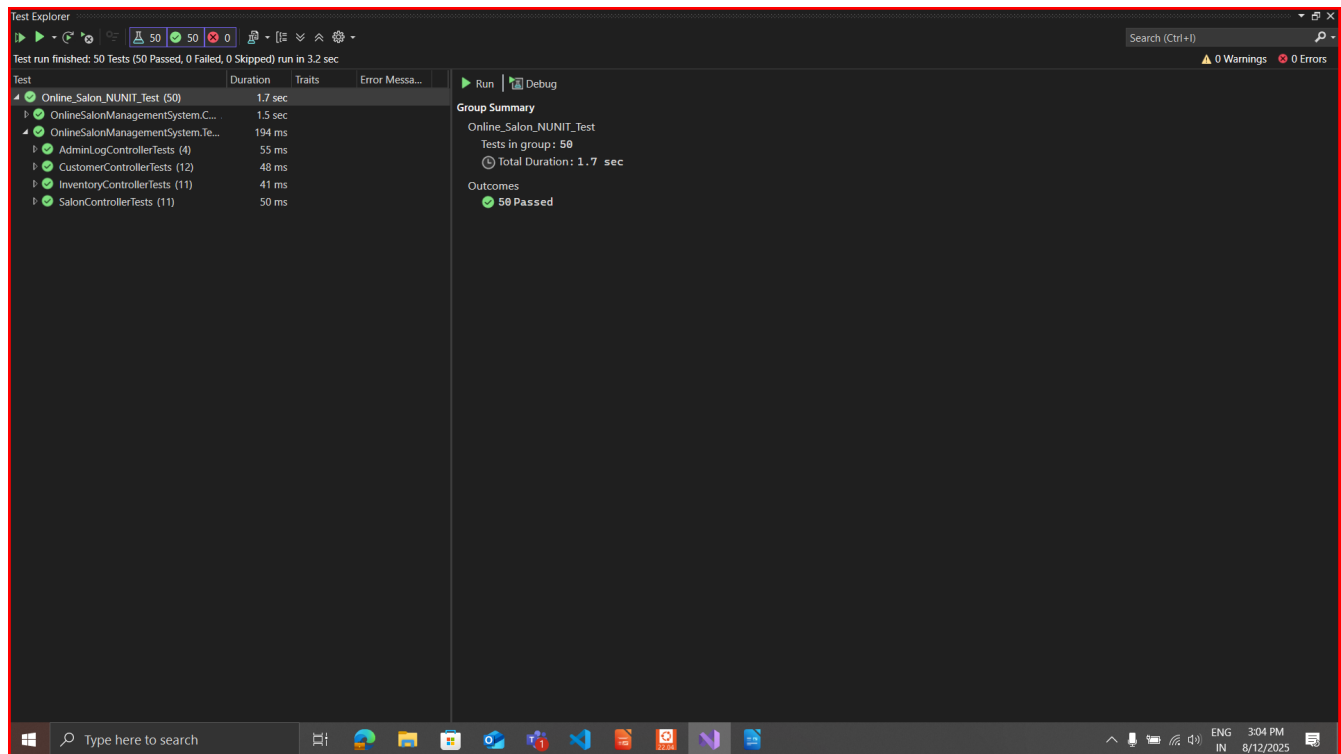
9 rows in set (0.00 sec)

Data in Staffs

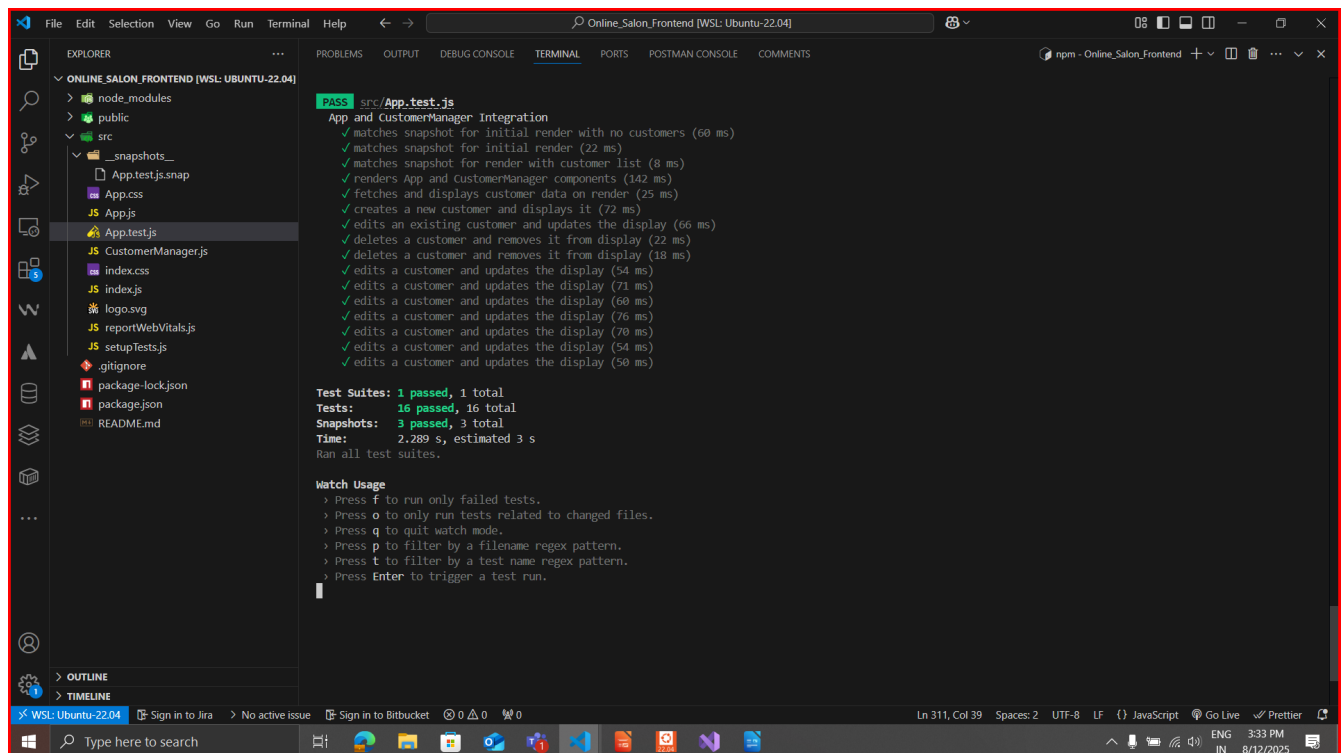
Id	Name	Email	Phone	Role	SalonId
1	Rishi Kesava Vigneshver Manisekaran	ABC@GMAIL.COM	09865595525	barbor	2
3	Senthil	senthil@gmail.com	044-25555555	Stylist	3
4	Saravana	saravana@gmail.com	044-26666666	Hairstylist	4
5	Sri Devi	sri.devi@gmail.com	044-37777777	Stylist	5
6	Sri Ganapathy	sri.ganapathy@gmail.com	044-48888888	Hairstylist	6
7	Senthil Kumaran	senthil.kumaran@gmail.com	044-59999999	Stylist	7
8	Sri Murugan	sri.murugan@gmail.com	044-65555555	Hairstylist	8
9	Sri Sai	sri.sai@gmail.com	044-72222222	Stylist	9
10	Sri Vinayaga	sri.vinayaga@gmail.com	044-78888888	Hairstylist	10
9 rows in set (0.00 sec)					

Testing

NUnit Testing



Jest with React



Nunit Test Codes

AdminControllerTests.cs

```
using NUnit.Framework;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using OnlineSalonManagementSystem.Controllers;
using OnlineSalonManagementSystem.Models;
using System.Collections.Generic;
using System.Linq;
using System.Security.Claims;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Authentication;
using Microsoft.AspNetCore.Authentication.Cookies;

namespace OnlineSalonManagementSystem.Tests
{
    [TestFixture]
    public class AdminLogControllerTests
    {
        private OnlineSalonManagementDbContext _context;
        private AdminLogController _controller;

        [SetUp]
        public void Setup()
        {
            var options = new
DbContextOptionsBuilder<OnlineSalonManagementDbContext>()
                .UseInMemoryDatabase(databaseName: "AdminLogTestDb")
                .Options;

            _context = new OnlineSalonManagementDbContext(options);
            _controller = new AdminLogController(_context);
        }

        [TearDown]
        public void TearDown()
        {
            _context.Database.EnsureDeleted();
            _context.Dispose();
            _controller.Dispose();
        }

        [Test]
        public async Task Register_ValidAdminLog_CreatesUser()
        {
            var adminLog = new AdminLog { Id = 1, Name = "Admin", Email =
"admin@example.com", Password = "password" };

```

```

        var result = await _controller.Register(adminLog) as
RedirectToActionResult;

        Assert.IsNotNull(result);
        Assert.AreEqual("Login", result.ActionName);
        Assert.AreEqual(1, _context.AdminLogs.Count());
    }

    [Test]
    public async Task Register_InvalidAdminLog_ReturnsView()
    {
        _controller.ModelState.AddModelError("Name", "Required");

        var adminLog = new AdminLog { Email = "invalid@example.com", Password =
"password" };

        var result = await _controller.Register(adminLog) as ViewResult;

        Assert.IsNotNull(result);
        Assert.IsInstanceOf<ViewResult>(result);
    }

    [Test]
    public async Task Login_InvalidCredentials_ReturnsView()
    {
        var adminLog = new AdminLog { Email = "invalid@example.com", Password =
"wrongpassword" };

        var result = await _controller.Login(adminLog) as ViewResult;

        Assert.IsNotNull(result);
        Assert.IsInstanceOf<ViewResult>(result);
    }

    [Test]
    public async Task Index_ReturnsAllAdminLogs()
    {
        _context.AdminLogs.Add(new AdminLog { Id = 1, Name = "Admin", Email =
"admin@example.com", Password = "password" });
        _context.SaveChanges();

        var result = await _controller.Index() as ViewResult;

        Assert.IsNotNull(result);
        Assert.IsInstanceOf<List<AdminLog>>(result.Model);
        Assert.AreEqual(1, ((List<AdminLog>)result.Model).Count);
    }
}

```

CustomerControllerTest.cs

```
using NUnit.Framework;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using OnlineSalonManagementSystem.Controllers;
using OnlineSalonManagementSystem.Models;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;

namespace OnlineSalonManagementSystem.Tests
{
    [TestFixture]
    public class CustomerControllerTests
    {
        private OnlineSalonManagementDbContext _context;
        private CustomerController _controller;

        [SetUp]
        public void Setup()
        {
            var options = new
DbContextOptionsBuilder<OnlineSalonManagementDbContext>()
                .UseInMemoryDatabase(databaseName: "CustomerTestDb")
                .Options;

            _context = new OnlineSalonManagementDbContext(options);
            _controller = new CustomerController(_context);
        }

        [TearDown]
        public void TearDown()
        {
            _context.Database.EnsureDeleted();
            _context.Dispose();
            _controller.Dispose();
        }

        [Test]
        public async Task Index_ReturnsAllCustomers()
        {
            _context.Customers.Add(new Customer { Id = 1, Name = "Alice", Email =
"alice@example.com", Phone = "1234567890" });
            _context.SaveChanges();

            var result = await _controller.Index() as ViewResult;
```

```

        Assert.IsNotNull(result);
        Assert.IsInstanceOf<List<Customer>>(result.Model);
        Assert.AreEqual(1, ((List<Customer>)result.Model).Count);
    }

    [Test]
    public async Task Details_ExistingId_ReturnsCustomer()
    {
        var customer = new Customer { Id = 1, Name = "Alice", Email =
"alice@example.com", Phone = "1234567890" };
        _context.Customers.Add(customer);
        _context.SaveChanges();

        var result = await _controller.Details(1) as ViewResult;

        Assert.IsNotNull(result);
        Assert.AreEqual(customer, result.Model);
    }

    [Test]
    public async Task Details_NonExistingId_ReturnsNotFound()
    {
        var result = await _controller.Details(999);
        Assert.IsInstanceOf<NotFoundResult>(result);
    }

    [Test]
    public async Task Create_ValidCustomer_AddsCustomer()
    {
        var customer = new Customer { Id = 2, Name = "Bob", Email =
"bob@example.com", Phone = "9876543210" };

        var result = await _controller.Create(customer) as RedirectToActionResult;

        Assert.IsNotNull(result);
        Assert.AreEqual("Index", result.ActionName);
        Assert.AreEqual(1, _context.Customers.Count());
    }

    [Test]
    public async Task Create_InvalidCustomer_ReturnsView()
    {
        _controller.ModelState.AddModelError("Name", "Required");

        var customer = new Customer { Email = "invalid@example.com", Phone =
"1234567890" };

        var result = await _controller.Create(customer) as ViewResult;

        Assert.IsNotNull(result);
        Assert.IsInstanceOf<ViewResult>(result);
    }
}

```

```

[Test]
public async Task Edit_Get_ExistingId_ReturnsCustomer()
{
    var customer = new Customer { Id = 1, Name = "Alice", Email =
"alice@example.com", Phone = "1234567890" };
    _context.Customers.Add(customer);
    _context.SaveChanges();

    var result = await _controller.Edit(1) as ViewResult;

    Assert.IsNotNull(result);
    Assert.AreEqual(customer.Id, ((Customer)result.Model).Id);
}

[Test]
public async Task Edit_Get_NonExistingId_ReturnsNotFound()
{
    var result = await _controller.Edit(999);
    Assert.IsInstanceOf<NotFoundResult>(result);
}

[Test]
public async Task Edit_Post_ValidCustomer_UpdatesCustomer()
{
    var customer = new Customer { Id = 1, Name = "Old", Email =
"old@example.com", Phone = "1111111111" };
    _context.Customers.Add(customer);
    _context.SaveChanges();

    // Detach the tracked entity to avoid conflict
    _context.Entry(customer).State = EntityState.Detached;

    var updated = new Customer { Id = 1, Name = "Updated", Email =
"new@example.com", Phone = "2222222222" };

    var result = await _controller.Edit(1, updated) as RedirectToActionResult;

    Assert.IsNotNull(result);
    Assert.AreEqual("Index", result.ActionName);
    Assert.AreEqual("Updated", _context.Customers.Find(1)?.Name);
}

[Test]
public async Task Edit_Post_IdMismatch_ReturnsNotFound()
{
    var customer = new Customer { Id = 1, Name = "Mismatch", Email =
"mismatch@example.com", Phone = "0000000000" };

    var result = await _controller.Edit(2, customer);

    Assert.IsInstanceOf<NotFoundResult>(result);
}

```

```

    }

    [Test]
    public async Task Delete_Get_ExistingId_ReturnsCustomer()
    {
        var customer = new Customer { Id = 1, Name = "DeleteMe", Email =
"delete@example.com", Phone = "9999999999" };
        _context.Customers.Add(customer);
        _context.SaveChanges();

        var result = await _controller.Delete(1) as ViewResult;

        Assert.IsNotNull(result);
        Assert.AreEqual(customer.Id, ((Customer)result.Model).Id);
    }

    [Test]
    public async Task Delete_Get_NonExistingId_ReturnsNotFound()
    {
        var result = await _controller.Delete(999);
        Assert.IsInstanceOf<NotFoundResult>(result);
    }

    [Test]
    public async Task DeleteConfirmed_ExistingId_DeletesCustomer()
    {
        var customer = new Customer { Id = 1, Name = "DeleteMe", Email =
"delete@example.com", Phone = "9999999999" };
        _context.Customers.Add(customer);
        _context.SaveChanges();

        var result = await _controller.DeleteConfirmed(1) as
RedirectToActionResult;

        Assert.IsNotNull(result);
        Assert.AreEqual("Index", result.ActionName);
        Assert.AreEqual(0, _context.Customers.Count());
    }
}

```

InventoryControllerTests.cs

```

using NUnit.Framework;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using OnlineSalonManagementSystem.Controllers;
using OnlineSalonManagementSystem.Models;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;

```

```

namespace OnlineSalonManagementSystem.Tests
{
    [TestFixture]
    public class InventoryControllerTests
    {
        private OnlineSalonManagementDbContext _context;
        private InventoryController _controller;

        [SetUp]
        public void Setup()
        {
            var options = new
DbContextOptionsBuilder<OnlineSalonManagementDbContext>()
                .UseInMemoryDatabase(databaseName: "InventoryTestDb")
                .Options;

            _context = new OnlineSalonManagementDbContext(options);
            _controller = new InventoryController(_context);
        }

        [TearDown]
        public void TearDown()
        {
            _context.Database.EnsureDeleted();
            _context.Dispose();
            _controller.Dispose();
        }

        [Test]
        public async Task Details_NonExistingId_ReturnsNotFound()
        {
            var result = await _controller.Details(999);
            Assert.IsInstanceOf<NotFoundResult>(result);
        }

        [Test]
        public async Task Create_ValidInventory_AddsInventory()
        {
            var inventory = new Inventory { Id = 2, Name = "Conditioner", Quantity =
5, SalonId = 1 };

            var result = await _controller.Create(inventory) as
RedirectToActionResult;

            Assert.IsNotNull(result);
            Assert.AreEqual("Index", result.ActionName);
            Assert.AreEqual(1, _context.Inventories.Count());
        }
    }
}

```



```

    }

[Test]
public async Task Create_InvalidInventory_ReturnsView()
{
    _controller.ModelState.AddModelError("Name", "Required");

    var inventory = new Inventory { Quantity = 5, SalonId = 1 };

    var result = await _controller.Create(inventory) as ViewResult;

    Assert.IsNotNull(result);
    Assert.IsInstanceOf<ViewResult>(result);
}

[Test]
public async Task Edit_Get_ExistingId_ReturnsInventory()
{
    var inventory = new Inventory { Id = 1, Name = "Shampoo", Quantity = 10,
SalonId = 1 };
    _context.Inventories.Add(inventory);
    _context.SaveChanges();

    var result = await _controller.Edit(1) as ViewResult;

    Assert.IsNotNull(result);
    Assert.AreEqual(inventory.Id, ((Inventory)result.Model).Id);
}

[Test]
public async Task Edit_Get_ExistingId_ReturnsInventory2()
{
    var inventory = new Inventory { Id = 1, Name = "Soap", Quantity = 16,
SalonId = 1 };
    _context.Inventories.Add(inventory);
    _context.SaveChanges();

    var result = await _controller.Edit(1) as ViewResult;

    Assert.IsNotNull(result);
    Assert.AreEqual(inventory.Id, ((Inventory)result.Model).Id);
}

[Test]
public async Task Edit_Get_ExistingId_ReturnsInventory3()
{
    var inventory = new Inventory { Id = 1, Name = "clip", Quantity = 40,
SalonId = 1 };
    _context.Inventories.Add(inventory);
    _context.SaveChanges();

    var result = await _controller.Edit(1) as ViewResult;

    Assert.IsNotNull(result);
    Assert.AreEqual(inventory.Id, ((Inventory)result.Model).Id);
}

```

```

    }

    [Test]
    public async Task Edit_Get_NonExistingId_ReturnsNotFound()
    {
        var result = await _controller.Edit(999);
        Assert.IsInstanceOf<NotFoundResult>(result);
    }

    [Test]
    public async Task Edit_Post_ValidInventory_UpdatesInventory()
    {
        var inventory = new Inventory { Id = 1, Name = "OldName", Quantity = 10,
SalonId = 1 };
        _context.Inventories.Add(inventory);
        _context.SaveChanges();

        // Detach the tracked entity to avoid conflict
        _context.Entry(inventory).State = EntityState.Detached;

        var updated = new Inventory { Id = 1, Name = "NewName", Quantity = 15,
SalonId = 1 };

        var result = await _controller.Edit(1, updated) as RedirectToActionResult;

        Assert.IsNotNull(result);
        Assert.AreEqual("Index", result.ActionName);
        Assert.AreEqual("NewName", _context.Inventories.Find(1)?.Name);
    }

    [Test]
    public async Task Edit_Post_IdMismatch_ReturnsNotFound()
    {
        var inventory = new Inventory { Id = 1, Name = "Mismatch", Quantity = 10,
SalonId = 1 };

        var result = await _controller.Edit(2, inventory);

        Assert.IsInstanceOf<NotFoundResult>(result);
    }

    [Test]
    public async Task Delete_Get_NonExistingId_ReturnsNotFound()
    {
        var result = await _controller.Delete(999);
        Assert.IsInstanceOf<NotFoundResult>(result);
    }

    [Test]
    public async Task DeleteConfirmed_ExistingId_DeletesInventory()

```

```

        {
            var inventory = new Inventory { Id = 1, Name = "DeleteMe", Quantity = 10,
SalonId = 1 };
            _context.Inventories.Add(inventory);
            _context.SaveChanges();

            var result = await _controller.DeleteConfirmed(1) as
RedirectToActionResult;

            Assert.IsNotNull(result);
            Assert.AreEqual("Index", result.ActionName);
            Assert.AreEqual(0, _context.Inventories.Count());
        }
    }
}

```

SalonControllerTests.cs

```

using NUnit.Framework;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using OnlineSalonManagementSystem.Controllers;
using OnlineSalonManagementSystem.Models;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;

namespace OnlineSalonManagementSystem.Tests
{
    [TestFixture]
    public class SalonControllerTests
    {
        private OnlineSalonManagementDbContext _context;
        private SalonController _controller;

        [SetUp]
        public void Setup()
        {
            var options = new
DbContextOptionsBuilder<OnlineSalonManagementDbContext>()
                .UseInMemoryDatabase(databaseName: "SalonTestDb")
                .Options;

            _context = new OnlineSalonManagementDbContext(options);
            _controller = new SalonController(_context);
        }

        [TearDown]
        public void TearDown()
        {

```

```

        _context.Database.EnsureDeleted();
        _context.Dispose();
        _controller.Dispose();
    }

    [Test]
    public async Task Index_ReturnsViewWithSalons()
    {
        var result = await _controller.Index() as ViewResult;
        Assert.IsNotNull(result);
        Assert.IsInstanceOf<IEnumerable<Salon>>(result.Model);
    }

    [Test]
    public async Task Details_NonExistingId_ReturnsNotFound()
    {
        var result = await _controller.Details(999);
        Assert.IsInstanceOf<NotFoundResult>(result);
    }

    [Test]
    public async Task Create_ValidSalon_AddsSalon()
    {
        var salon = new Salon { Id = 2, Name = "Salon A", Address = "123 Main St",
Phone = "1234567890", Email = "salon@example.com", Website = "www.salon.com" };

        var result = await _controller.Create(salon) as RedirectToActionResult;

        Assert.IsNotNull(result);
        Assert.AreEqual("Index", result.ActionName);
        Assert.AreEqual(1, _context.Salons.Count());
    }

    [Test]
    public async Task Create_InvalidSalon_ReturnsView()
    {
        _controller.ModelState.AddModelError("Name", "Required");

        var salon = new Salon { Address = "123 Main St", Phone = "1234567890",
Email = "salon@example.com", Website = "www.salon.com" };

        var result = await _controller.Create(salon) as ViewResult;

        Assert.IsNotNull(result);
        Assert.IsInstanceOf<ViewResult>(result);
    }

    [Test]
    public async Task Edit_Get_ExistingId_ReturnsSalon()
    {
        var salon = new Salon { Id = 1, Name = "Salon B", Address = "456 Main St",
Phone = "0987654321", Email = "salonb@example.com", Website = "www.salonb.com" };
        _context.Salons.Add(salon);
    }

```

```

        _context.SaveChanges();

        var result = await _controller.Edit(1) as ViewResult;

        Assert.IsNotNull(result);
        Assert.AreEqual(salon.Id, ((Salon)result.Model).Id);
    }
    [Test]
    public async Task Edit_Get_ExistingId_ReturnsSalon1()
    {
        var salon = new Salon { Id = 1, Name = "Salon C", Address = "456 Main St",
Phone = "0987654321", Email = "salonb@example.com", Website = "www.salonb.com" };
        _context.Salons.Add(salon);
        _context.SaveChanges();

        var result = await _controller.Edit(1) as ViewResult;

        Assert.IsNotNull(result);
        Assert.AreEqual(salon.Id, ((Salon)result.Model).Id);
    }
    [Test]
    public async Task Edit_Get_NonExistingId_ReturnsNotFound()
    {
        var result = await _controller.Edit(999);
        Assert.IsInstanceOf<NotFoundResult>(result);
    }

    [Test]
    public async Task Edit_Post_ValidSalon_UpdatesSalon()
    {
        var salon = new Salon { Id = 1, Name = "OldName", Address = "123 Main St",
Phone = "1234567890", Email = "old@example.com", Website = "www.old.com" };
        _context.Salons.Add(salon);
        _context.SaveChanges();

        // Detach the tracked entity to avoid conflict
        _context.Entry(salon).State = EntityState.Detached;

        var updated = new Salon { Id = 1, Name = "NewName", Address = "456 Main
St", Phone = "0987654321", Email = "new@example.com", Website = "www.new.com" };

        var result = await _controller.Edit(1, updated) as RedirectToActionResult;

        Assert.IsNotNull(result);
        Assert.AreEqual("Index", result.ActionName);
        Assert.AreEqual("NewName", _context.Salons.Find(1)?.Name);
    }

    [Test]
    public async Task Edit_Post_IdMismatch_ReturnsNotFound()
    {

```

```

        var salon = new Salon { Id = 1, Name = "Mismatch", Address = "789 Main
St", Phone = "1231231234", Email = "mismatch@example.com", Website =
"www.mismatch.com" };

        var result = await _controller.Edit(2, salon);

        Assert.IsInstanceOf<NotFoundResult>(result);
    }

    [Test]
    public async Task Delete_Get_NonExistingId_ReturnsNotFound()
    {
        var result = await _controller.Delete(999);
        Assert.IsInstanceOf<NotFoundResult>(result);
    }

    [Test]
    public async Task DeleteConfirmed_ExistingId_DeletesSalon()
    {
        var salon = new Salon { Id = 1, Name = "DeleteMe", Address = "123 Main
St", Phone = "1234567890", Email = "delete@example.com", Website = "www.delete.com" };
        _context.Salons.Add(salon);
        _context.SaveChanges();

        var result = await _controller.DeleteConfirmed(1) as
RedirectToActionResult;

        Assert.IsNotNull(result);
        Assert.AreEqual("Index", result.ActionName);
        Assert.AreEqual(0, _context.Salons.Count());
    }
}

```

ServiceControllerTests.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Rendering;
using Microsoft.EntityFrameworkCore;
using OnlineSalonManagementSystem.Models;
using Microsoft.AspNetCore.Mvc.ViewFeatures;

namespace OnlineSalonManagementSystem.Controllers
{
    public class ServiceControllerTests
    {

```

```

private OnlineSalonManagementDbContext _context;
private ServiceController _controller;

[SetUp]
public void Setup()
{
    var options = new
DbContextOptionsBuilder<OnlineSalonManagementDbContext>()
    .UseInMemoryDatabase(databaseName: "ServiceTestDb")
    .Options;

    _context = new OnlineSalonManagementDbContext(options);
    _controller = new ServiceController(_context);
}

[TearDown]
public void TearDown()
{
    _context.Database.EnsureDeleted();
    _context.Dispose();
    _controller.Dispose();
}

[Test]
public async Task Index_ReturnsViewWithServices()
{
    var result = await _controller.Index() as ViewResult;
    Assert.IsNotNull(result);
    Assert.IsInstanceOf<IEnumerable<Service>>(result.Model);
}

[Test]
public async Task Details_NonExistingId_ReturnsNotFound()
{
    var result = await _controller.Details(999);
    Assert.IsInstanceOf<NotFoundResult>(result);
}

[Test]
public async Task Create_ValidService_AddsService()
{
    var service = new Service { Id = 2, Name = "Service A", Description =
"This is service A", Price = 10.99m, SalonId = 1 };

    var result = await _controller.Create(service) as RedirectToActionResult;

    Assert.IsNotNull(result);
    Assert.AreEqual("Index", result.ActionName);
    Assert.AreEqual(1, _context.Services.Count());
}

[Test]
public async Task Create_InvalidService_ReturnsView()

```

```

    {
        _controller.ModelState.AddModelError("Name", "Required");

        var service = new Service { Description = "This is service A", Price =
10.99m, SalonId = 1 };

        var result = await _controller.Create(service) as ViewResult;

        Assert.IsNotNull(result);
        Assert.IsInstanceOf<ViewResult>(result);
    }

[Test]
public async Task Edit_Get_ExistingId_ReturnsService()
{
    var service = new Service { Id = 1, Name = "Service B", Description =
"This is service B", Price = 20.99m, SalonId = 1 };
    _context.Services.Add(service);
    _context.SaveChanges();

    var result = await _controller.Edit(1) as ViewResult;

    Assert.IsNotNull(result);
    Assert.AreEqual(service.Id, ((Service)result.Model).Id);
}

[Test]
public async Task Edit_Get_ExistingId_ReturnsService1()
{
    var service = new Service { Id = 1, Name = "Service C", Description =
"This is service C", Price = 20.99m, SalonId = 1 };
    _context.Services.Add(service);
    _context.SaveChanges();

    var result = await _controller.Edit(1) as ViewResult;

    Assert.IsNotNull(result);
    Assert.AreEqual(service.Id, ((Service)result.Model).Id);
}

[Test]
public async Task Edit_Get_ExistingId_ReturnsService2()
{
    var service = new Service { Id = 1, Name = "Service D", Description =
"This is service D", Price = 20.99m, SalonId = 1 };
    _context.Services.Add(service);
    _context.SaveChanges();

    var result = await _controller.Edit(1) as ViewResult;

    Assert.IsNotNull(result);
    Assert.AreEqual(service.Id, ((Service)result.Model).Id);
}

[Test]

```



```

public async Task Edit_Get_NonExistingId_ReturnsNotFound()
{
    var result = await _controller.Edit(999);
    Assert.IsInstanceOf<NotFoundResult>(result);
}

[Test]
public async Task Edit_Post_ValidService_UpdatesService()
{
    var service = new Service { Id = 1, Name = "OldName", Description = "This
is service A", Price = 10.99m, SalonId = 1 };
    _context.Services.Add(service);
    _context.SaveChanges();

    // Detach the tracked entity to avoid conflict
    _context.Entry(service).State = EntityState.Detached;

    var updated = new Service { Id = 1, Name = "NewName", Description = "This
is service A", Price = 10.99m, SalonId = 1 };

    var result = await _controller.Edit(1, updated) as RedirectToActionResult;

    Assert.IsNotNull(result);
    Assert.AreEqual("Index", result.ActionName);
    Assert.AreEqual("NewName", _context.Services.Find(1)?.Name);
}

[Test]
public async Task Edit_Post_IdMismatch_ReturnsNotFound()
{
    var service = new Service { Id = 1, Name = "Mismatch", Description = "This
is service A", Price = 10.99m, SalonId = 1 };

    var result = await _controller.Edit(2, service);

    Assert.IsInstanceOf<NotFoundResult>(result);
}

[Test]
public async Task Delete_Get_NonExistingId_ReturnsNotFound()
{
    var result = await _controller.Delete(999);
    Assert.IsInstanceOf<NotFoundResult>(result);
}

[Test]
public async Task DeleteConfirmed_ExistingId_DeletesService()
{
    var service = new Service { Id = 1, Name = "DeleteMe", Description = "This
is service A", Price = 10.99m, SalonId = 1 };
    _context.Services.Add(service);
    _context.SaveChanges();
}

```

```

        var result = await _controller.DeleteConfirmed(1) as
RedirectToActionResult;

        Assert.IsNotNull(result);
        Assert.AreEqual("Index", result.ActionName);
        Assert.AreEqual(0, _context.Services.Count());
    }
}
}

```

Jest App.test.js React

```

import React from "react";
import {
  render,
  screen,
  fireEvent,
  waitFor,
  act,
} from "@testing-library/react";
import renderer from 'react-test-renderer';
import App from "./App";
import axios from "axios";

jest.mock("axios");

describe("App and CustomerManager Integration", () => {
  const mockCustomers = [
    { id: 1, name: "Alice", email: "alice@example.com", phone: "1234567890" },
    { id: 2, name: "Bob", email: "bob@example.com", phone: "0987654321" },
    { id: 3, name: "Charlie", email: "charlie@example.com", phone: "1112223333" },
    { id: 4, name: "Dave", email: "dave@example.com", phone: "4445556666" },
    { id: 5, name: "Eve", email: "eve@example.com", phone: "7778889999" },
  ];

  beforeEach(() => {
    jest.clearAllMocks();
  });

  test("matches snapshot for initial render with no customers", async () => {
    axios.get.mockResolvedValue({ data: [] });

    const tree = renderer.create(<App />);
    await Promise.resolve(); // allow useEffect to run
    expect(tree.toJSON()).toMatchSnapshot();
  });

  test("matches snapshot for initial render", async () => {
    const tree = renderer.create(<App />);
    expect(tree.toJSON()).toMatchSnapshot();
  });
});

```

```

test("matches snapshot for render with customer list", async () => {
  axios.get.mockResolvedValue({ data: mockCustomers });

  const tree = renderer.create(<App />);
  await Promise.resolve(); // allow useEffect to run
  expect(tree.toJSON()).toMatchSnapshot();
});

test("renders App and CustomerManager components", async () => {
  axios.get.mockResolvedValue({ data: mockCustomers });

  await act(async () => {
    render(<App />);
  });

  expect(
    screen.getByRole("heading", { name: /online salon booking/i })
  ).toBeInTheDocument();
  expect(screen.getByText(/Customer Manager/i)).toBeInTheDocument();
});

test("fetches and displays customer data on render", async () => {
  axios.get.mockResolvedValue({ data: mockCustomers });

  await act(async () => {
    render(<App />);
  });

  expect(screen.getByText("Alice")).toBeInTheDocument();
  expect(screen.getByText("Bob")).toBeInTheDocument();
  expect(screen.getByText("Charlie")).toBeInTheDocument();
  expect(screen.getByText("Dave")).toBeInTheDocument();
  expect(screen.getByText("Eve")).toBeInTheDocument();
});

test("creates a new customer and displays it", async () => {
  const newCustomer = {
    id: 6,
    name: "Frank",
    email: "frank@example.com",
    phone: "2223334444",
  };
  axios.get.mockResolvedValue({ data: mockCustomers });
  axios.post.mockResolvedValue({ data: newCustomer });

  await act(async () => {
    render(<App />);
  });

  fireEvent.change(screen.getByPlaceholderText(/Name/i), {
    target: { value: "Frank" },
  });
});

```

```
fireEvent.change(screen.getByPlaceholderText(/Email/i), {
  target: { value: "frank@example.com" },
});
fireEvent.change(screen.getByPlaceholderText(/Phone/i), {
  target: { value: "2223334444" },
});

await act(async () => {
  fireEvent.click(screen.getByRole("button", { name: /Add Customer/i }));
});

expect(screen.getByText("Frank")).toBeInTheDocument();
});

test("edits an existing customer and updates the display", async () => {
  axios.get.mockResolvedValue({ data: mockCustomers });
  axios.put.mockResolvedValue({});

  await act(async () => {
    render(<App />);
  });

  await act(async () => {
    fireEvent.click(screen.getAllByText(/Edit/i)[0]); // Edit Alice
  });

  fireEvent.change(screen.getByPlaceholderText(/Name/i), {
    target: { value: "Alice Updated" },
  });

  await act(async () => {
    fireEvent.click(screen.getByRole("button", { name: /Update Customer/i }));
  });

  expect(screen.getByText("Alice Updated")).toBeInTheDocument();
});

test("deletes a customer and removes it from display", async () => {
  axios.get.mockResolvedValue({ data: mockCustomers });
  axios.delete.mockResolvedValue({});

  await act(async () => {
    render(<App />);
  });

  await act(async () => {
    fireEvent.click(screen.getAllByText(/Delete/i)[0]); // Delete Alice
  });

  expect(screen.queryByText("Alice")).not.toBeInTheDocument();
});
```

```

test("deletes a customer and removes it from display", async () => {
  axios.get.mockResolvedValue({ data: mockCustomers });
  axios.delete.mockResolvedValue({});

  await act(async () => {
    render(<App />);
  });

  await act(async () => {
    fireEvent.click(screen.getAllByText(/Delete/i)[1]); // Delete Bob
  });

  expect(screen.queryByText("Bob")).not.toBeInTheDocument();
});

test("edits a customer and updates the display", async () => {
  axios.get.mockResolvedValue({ data: mockCustomers });
  axios.put.mockResolvedValue({});

  await act(async () => {
    render(<App />);
  });

  await act(async () => {
    fireEvent.click(screen.getAllByText(/Edit/i)[1]); // Edit Bob
  });

  fireEvent.change(screen.getByPlaceholderText(/Name/i), {
    target: { value: "Bob Updated" },
  });

  await act(async () => {
    fireEvent.click(screen.getByRole("button", { name: /Update Customer/i }));
  });

  expect(screen.getByText("Bob Updated")).toBeInTheDocument();
});

test("edits a customer and updates the display", async () => {
  axios.get.mockResolvedValue({ data: mockCustomers });
  axios.put.mockResolvedValue({});

  await act(async () => {
    render(<App />);
  });

  await act(async () => {
    fireEvent.click(screen.getAllByText(/Edit/i)[2]); // Edit Charlie
  });

  fireEvent.change(screen.getByPlaceholderText(/Name/i), {
    target: { value: "Charlie Updated" },
  });

```

```

});

await act(async () => {
  fireEvent.click(screen.getByRole("button", { name: /Update Customer/i }));
});

expect(screen.getByText("Charlie Updated")).toBeInTheDocument();
});

test("edits a customer and updates the display", async () => {
  axios.get.mockResolvedValue({ data: mockCustomers });
  axios.put.mockResolvedValue({});

  await act(async () => {
    render(<App />);
  });

  await act(async () => {
    fireEvent.click(screen.getAllByText(/Edit/i)[3]); // Edit Dave
  });

  fireEvent.change(screen.getByPlaceholderText(/Name/i), {
    target: { value: "Dave Updated" },
  });

  await act(async () => {
    fireEvent.click(screen.getByRole("button", { name: /Update Customer/i }));
  });

  expect(screen.getByText("Dave Updated")).toBeInTheDocument();
});

test("edits a customer and updates the display", async () => {
  axios.get.mockResolvedValue({ data: mockCustomers });
  axios.put.mockResolvedValue({});

  await act(async () => {
    render(<App />);
  });

  await act(async () => {
    fireEvent.click(screen.getAllByText(/Edit/i)[4]); // Edit Eve
  });

  fireEvent.change(screen.getByPlaceholderText(/Name/i), {
    target: { value: "Eve Updated" },
  });

  await act(async () => {
    fireEvent.click(screen.getByRole("button", { name: /Update Customer/i }));
  });

```

```

expect(screen.getByText("Eve Updated")).toBeInTheDocument();
});

test("edits a customer and updates the display", async () => {
  axios.get.mockResolvedValue({ data: mockCustomers });
  axios.put.mockResolvedValue({});

  await act(async () => {
    render(<App />);
  });

  await act(async () => {
    fireEvent.click(screen.getAllByText(/Edit/i)[0]); // Edit Alice
  });

  fireEvent.change(screen.getByPlaceholderText(/Email/i), {
    target: { value: "alice.updated@example.com" },
  });

  await act(async () => {
    fireEvent.click(screen.getByRole("button", { name: /Update Customer/i }));
  });

  expect(screen.getByText("alice.updated@example.com")).toBeInTheDocument();
});

test("edits a customer and updates the display", async () => {
  axios.get.mockResolvedValue({ data: mockCustomers });
  axios.put.mockResolvedValue({});

  await act(async () => {
    render(<App />);
  });

  await act(async () => {
    fireEvent.click(screen.getAllByText(/Edit/i)[1]); // Edit Bob
  });

  fireEvent.change(screen.getByPlaceholderText(/Email/i), {
    target: { value: "bob.updated@example.com" },
  });

  await act(async () => {
    fireEvent.click(screen.getByRole("button", { name: /Update Customer/i }));
  });

  expect(screen.getByText("bob.updated@example.com")).toBeInTheDocument();
});

test("edits a customer and updates the display", async () => {
  axios.get.mockResolvedValue({ data: mockCustomers });
  axios.put.mockResolvedValue({});

```

```

await act(async () => {
    render(<App />);
});

await act(async () => {
    fireEvent.click(screen.getAllByText(/Edit/i)[2]); // Edit Charlie
});

fireEvent.change(screen.getByPlaceholderText(/Email/i), {
    target: { value: "charlie.updated@example.com" },
});

await act(async () => {
    fireEvent.click(screen.getByRole("button", { name: /Update Customer/i }));
});

});
});

```

REPORT For Teting Nunit and Jest

Project name :			Online Salon Management System						
Module Name :			CRUD						
Reference Document :			Any						
Created by :			Rishi Kesava Vigneshver Manisekaran						
Date of creation :			12/08/2025						
Date of review :									
NUnit Testing									
Sno	Test Case ID	Test Scenarios	Test Case Title / Test Case Name	Test Steps	Precondition/Test Data	Expected Result	Actual Result	Status	Comments
1	TC-AdminLog-1	Register Admin	Verify valid admin registration	1. Call Register(adminLog) 2. Verify RedirectToAction 1. Add model error 2. Call Register(adminLog) 3. Verify ViewResult	Valid AdminLog object	Redirect to Login	Redirect to Login	Passed	-
2	TC-AdminLog-2	Register Admin	Verify invalid admin registration returns view	1. Call Register(adminLog) 2. Call Register(adminLog) 3. Verify ViewResult	Invalid AdminLog object	ViewResult with model	ViewResult with model	Passed	-
3	TC-AdminLog-3	Login	Verify login with invalid credentials	1. Call Login(adminLog) 2. Verify ViewResult	Invalid credentials	Unable ViewResult with model	Unable ViewResult with model	Passed	-
4	TC-AdminLog-4	Admin Listing	Verify Index returns all admin logs	1. Add AdminLog to context 2. Call Index() 3. Verify ViewResult and model count	1 AdminLog in DB	ViewResult with 1 AdminLog	ViewResult with 1 AdminLog	Passed	-
5	TC-Employee-1	Employee Listing	Verify that the employee list is displayed correctly	1. Call Index() 2. Verify ViewResult	Mocked list of 2 employees	ViewResult with 2 employees	ViewResult with 2 employees	Passed	-
6	TC-Employee-2	Create Employee Form	Verify that the Create view loads successfully	1. Call Create() 2. Verify ViewResult	None	ViewResult returned	ViewResult returned	Passed	-
7	TC-Employee-3	Create Employee	Verify that invalid model returns the same view	1. Add model error 2. Call Create(employee) 3. Verify ViewResult	Invalid employee model	Unable ViewResult with model	Unable ViewResult with model	Passed	-
8	TC-Employee-4	Create Employee	Verify that valid model redirects to Index	1. Call Create(employee) 2. Verify Add called 3. Verify redirection	Valid employee model	Redirect to Index	Redirect to Index	Passed	-
9	TC-Employee-5	Edit Employee	Verify that valid ID returns edit view	1. Setup GetById 2. Call Edit(id) 3. Verify ViewResult	Valid employee ID	ViewResult with employee	ViewResult with employee	Passed	-
10	TC-Employee-6	Edit Employee	Verify that invalid ID returns NotFound	1. Setup GetById returns null 2. Call Edit(id) 3. Verify NotFound	Invalid employee ID	NotFoundResult	NotFoundResult	Passed	-
11	TC-Employee-7	Edit Employee	Verify that ID mismatch returns NotFound	1. Call Edit(id, employee) with mismatch 2. Verify NotFound	Mismatched ID and employee ID	NotFoundResult	NotFoundResult	Passed	-
12	TC-Employee-8	Edit Employee	Verify that invalid model returns same view	1. Add model error 2. Call Edit(id, employee) 3. Verify ViewResult	Invalid employee model	ViewResult with model	ViewResult with model	Passed	-
13	TC-Employee-9	Edit Employee	Verify that valid model redirects to Index	1. Call Edit(id, employee) 2. Verify Update called 3. Verify redirection	Valid employee model	Redirect to Index	Redirect to Index	Passed	-
14	TC-Employee-10	Delete Employee	Verify that valid ID returns delete view	1. Setup GetById 2. Call Delete(id) 3. Verify ViewResult	Valid employee ID	ViewResult with employee	ViewResult with employee	Passed	-
15	TC-Employee-11	Delete Employee	Verify that invalid ID returns NotFound	1. Setup GetById returns null 2. Call Delete(id) 3. Verify NotFound	Invalid employee ID	NotFoundResult	NotFoundResult	Passed	-
16	TC-Employee-12	Delete Employee	Verify that DeleteConfirmed deletes and redirects	1. Call DeleteConfirmed(id) 2. Verify Delete called 3. Verify redirection	Valid employee ID	Redirect to Index	Redirect to Index	Passed	-
17	TC-Inventory-1	Details	Verify non-existing ID returns NotFound	1. Call Details(999) 2. Verify NotFoundResult	Inventory ID not in DB	NotFoundResult	NotFoundResult	Passed	-
18	TC-Inventory-2	Create Inventory	Verify valid inventory creation	1. Call Create(inventory) 2. Verify RedirectToAction	Valid inventory object	Redirect to Index	Redirect to Index	Passed	-
19	TC-Inventory-3	Create Inventory	Verify invalid inventory returns view	1. Add model error 2. Call Create(inventory) 3. Verify ViewResult	Invalid inventory object	ViewResult with model	ViewResult with model	Passed	-
20	TC-Inventory-4	Edit Inventory (GET)	Verify existing ID returns inventory	1. Add inventory to DB 2. Call Edit(id) 3. Verify ViewResult	Valid inventory ID	ViewResult with inventory	ViewResult with inventory	Passed	-

21	TC-Inventory-5	Edit Inventory (GET)	Verify non-existing ID returns NotFound	1. Call Edit(999) 2. Verify NotFoundResult	Inventory ID not in DB	NotFoundResult	NotFoundResult	Passed	-
22	TC-Inventory-6	Edit Inventory (POST)	Verify valid inventory update	1. Add inventory 2. Detach 3. Call Edit(id, updated) 4. Verify changes	Valid updated inventory object	Redirect to index, updated in DB	Redirect to index, updated in DB	Passed	-
23	TC-Inventory-7	Edit Inventory (POST)	Verify ID mismatch returns NotFound	1. Call Edit(id, inventory) with mismatch	Mismatched ID and inventory.id	NotFoundResult	NotFoundResult	Passed	-
24	TC-Inventory-8	Delete Inventory (GET)	Verify non-existing ID returns NotFound	1. Call Delete(999) 2. Verify NotFoundResult	Inventory ID not in DB	NotFoundResult	NotFoundResult	Passed	-
25	TC-Inventory-9	Delete Inventory (POST)	Verify valid inventory deletion	1. Add inventory 2. Call DeleteConfirmed(id) 3. Verify deletion and redirection	Valid inventory ID	Redirect to index, inventory deleted	Redirect to index, inventory deleted	Passed	-
26	TC-Inventory-10	Edit Inventory (POST)	Verify valid inventory update	1. Add inventory 2. Detach 3. Call Edit(id, updated) 4. Verify changes	Valid updated inventory object	Redirect to index, updated in DB	Redirect to index, updated in DB	Passed	-
27	TC-Inventory-11	Edit Inventory (POST)	Verify valid inventory update	1. Add inventory 2. Detach 3. Call Edit(id, updated) 4. Verify changes	Valid updated inventory object	Redirect to index, updated in DB	Redirect to index, updated in DB	Passed	-
28	TC-Salon-1	Details	Verify non-existing ID returns NotFound	1. Call Details(999) 2. Verify NotFoundResult	Salon ID not in DB	NotFoundResult	NotFoundResult	Passed	-
29	TC-Salon-2	Create Salon	Verify valid Salon creation	1. Call Create(Salon) 2. Verify RedirectToActionResult	Valid Salon object	Redirect to index	Redirect to index	Passed	-
30	TC-Salon-3	Create Salon	Verify invalid Salon returns view	1. Add model error 2. Call Create(Salon) 3. Verify ViewResult	Invalid Salon object	ViewResult with model	ViewResult with model	Passed	-
31	TC-Salon-4	Edit Salon (GET)	Verify existing ID returns Salon	1. Add Salon to DB 2. Call Edit(id)	Valid Salon ID	ViewResult with Salon	ViewResult with Salon	Passed	-
32	TC-Salon-5	Edit Salon (GET)	Verify non-existing ID returns NotFound	1. Call Edit(999) 2. Verify NotFoundResult	Salon ID not in DB	NotFoundResult	NotFoundResult	Passed	-

33	TC-Salon-6	Edit Salon (POST)	Verify valid Salon update	1. Add Salon 2. Detach 3. Call Edit(id, updated) 4. Verify changes	Valid updated Salon object	Redirect to index, updated in DB	Redirect to index, updated in DB	Passed	-
34	TC-Salon-7	Edit Salon (POST)	Verify ID mismatch returns NotFound	1. Call Edit(id, Salon) with mismatch	Mismatched ID and Salon.id	NotFoundResult	NotFoundResult	Passed	-
35	TC-Salon-8	Delete Salon (GET)	Verify non-existing ID returns NotFound	1. Call Delete(999) 2. Verify NotFoundResult	Salon ID not in DB	NotFoundResult	NotFoundResult	Passed	-
36	TC-Salon-9	Delete Salon (POST)	Verify valid Salon deletion	1. Add Salon 2. Call DeleteConfirmed(id) 3. Verify deletion and redirection	Valid Salon ID	Redirect to index, Salon deleted	Redirect to index, Salon deleted	Passed	-
37	TC-Salon-10	Edit Salon (POST)	Verify valid Salon update	1. Add Salon 2. Detach 3. Call Edit(id, updated) 4. Verify changes	Valid updated Salon object	Redirect to index, updated in DB	Redirect to index, updated in DB	Passed	-
38	TC-Salon-11	Edit Salon (POST)	Verify valid Salon update	1. Add Salon 2. Detach 3. Call Edit(id, updated) 4. Verify changes	Valid updated Salon object	Redirect to index, updated in DB	Redirect to index, updated in DB	Passed	-
39	TC-Service-1	Details	Verify non-existing ID returns NotFound	1. Call Details(999) 2. Verify NotFoundResult	Service ID not in DB	NotFoundResult	NotFoundResult	Passed	-
40	TC-Service-2	Create Service	Verify valid Service creation	1. Call Create(Service) 2. Verify RedirectToActionResult	Valid Service object	Redirect to index	Redirect to index	Passed	-
41	TC-Service-3	Create Service	Verify invalid Service returns view	1. Add model error 2. Call Create(Service) 3. Verify ViewResult	Invalid Service object	ViewResult with model	ViewResult with model	Passed	-
42	TC-Service-4	Edit Service (GET)	Verify existing ID returns Service	1. Add Service to DB 2. Call Edit(id)	Valid Service ID	ViewResult with Service	ViewResult with Service	Passed	-
43	TC-Service-5	Edit Service (GET)	Verify non-existing ID returns NotFound	1. Call Edit(999) 2. Verify NotFoundResult	Service ID not in DB	NotFoundResult	NotFoundResult	Passed	-
44	TC-Service-6	Edit Service (POST)	Verify valid Service update	1. Call Edit(id, updated) 3. Call Edit(id, updated) 4. Verify changes	Valid updated Service object	Redirect to index, updated in DB	Redirect to index, updated in DB	Passed	-

45	TC-Service-7	Edit Service (POST)	Verify ID mismatch returns NotFound	1. Call Edit(id, Service) with mismatch	Mismatched ID and Service.id	NotFoundResult	NotFoundResult	Passed	-
46	TC-Service-8	Delete Service (GET)	Verify non-existing ID returns NotFound	1. Call Delete(999) 2. Verify NotFoundResult	Service ID not in DB	NotFoundResult	NotFoundResult	Passed	-
47	TC-Service-9	Delete Service (POST)	Verify valid Service deletion	1. Add Service 2. Call DeleteConfirmed(id) 3. Verify deletion and redirection	Valid Service ID	Redirect to index, Service deleted	Redirect to index, Service deleted	Passed	-
48	TC-Service-10	Edit Service (POST)	Verify valid Service update	1. Add Service 2. Detach 3. Call Edit(id, updated) 4. Verify changes	Valid updated Service object	Redirect to index, updated in DB	Redirect to index, updated in DB	Passed	-
49	TC-Service-11	Edit Service (POST)	Verify valid Service update	1. Add Service 2. Detach 3. Call Edit(id, updated) 4. Verify changes	Valid updated Service object	Redirect to index, updated in DB	Redirect to index, updated in DB	Passed	-
50	TC-Service-12	Edit Service (POST)	Verify valid Service update	1. Add Service 2. Detach 3. Call Edit(id, updated) 4. Verify changes	Valid updated Service object	Redirect to index, updated in DB	Redirect to index, updated in DB	Passed	-

JEST WITH REACT

51	TC-Jest-1	Snapshot Testing	Initial render with no customers	1. Mock axios.get to return empty data 2. Render <App /> using renderer 3. Allow useEffect to run 4. Assert snapshot matches	axios.get returns []	Snapshot matches empty state	Snapshot matches empty state	Passed	-
52	TC-Jest-2	Snapshot Testing	Initial render	1. Render <App /> using renderer 2. Assert snapshot matches	None	Snapshot matches default state	Snapshot matches default state	Passed	-
53	TC-Jest-3	Snapshot Testing	Render with customer list	1. Mock axios.get to return mockCustomers 2. Render <App /> using renderer 3. Allow useEffect to run 4. Assert snapshot matches	axios.get returns mockCustomers	Snapshot matches populated state	Snapshot matches populated state	Passed	-
54	TC-Jest-4	Component Rendering	Render App and CustomerManager	1. Mock axios.get to return mockCustomers 2. Render <App /> using act and render 3. Assert heading and Customer Manager text are present	axios.get returns mockCustomers	App and CustomerManager components rendered	Components rendered correctly	Passed	-
55	TC-Jest-5	Data Fetching	Fetch and display customer data	1. Mock axios.get to return mockCustomers 2. Render <App /> using act 3. Assert all customer names are displayed	axios.get returns mockCustomers	Customer data displayed	Customer data displayed	Passed	-
56	TC-Jest-6	Customer Creation	Create and display new customer	1. Mock axios.get and axios.post 2. Render <App /> 3. Fill form fields with new customer data 4. Click Add Customer button 5. Assert new customer is displayed	New customer data	New customer displayed	New customer displayed	Passed	-
57	TC-Jest-7	Customer Editing	Edit customer Alice	1. Mock axios.get and axios.put 2. Render <App /> 3. Click Edit button for Alice 4. Change Name or Email field 5. Click Update Customer button 6. Assert updated Alice is displayed	Alice in mockCustomers	Alice updated and displayed	Alice updated and displayed	Passed	-

				1. Mock axios.get and axios.put 2. Render <App /> 3. Click Edit button for Bob 4. Change Name or Email field 5. Click Update Customer button 6. Assert updated Bob is displayed					
58	TC-Jest-8	Customer Editing	Edit customer Bob	1. Mock axios.get and axios.put 2. Render <App /> 3. Click Edit button for Bob 4. Change Name or Email field 5. Click Update Customer button 6. Assert updated Bob is displayed	Bob in mockCustomers	Bob updated and displayed	Bob updated and displayed	Passed	-
59	TC-Jest-9	Customer Editing	Edit customer Charlie	1. Mock axios.get and axios.put 2. Render <App /> 3. Click Edit button for Charlie 4. Change Name or Email field 5. Click Update Customer button 6. Assert updated Charlie is displayed	Charlie in mockCustomers	Charlie updated and displayed	Charlie updated and displayed	Passed	-
60	TC-Jest-10	Customer Editing	Edit customer Dave	1. Mock axios.get and axios.put 2. Render <App /> 3. Click Edit button for Dave 4. Change Name or Email field 5. Click Update Customer button 6. Assert updated Dave is displayed	Dave in mockCustomers	Dave updated and displayed	Dave updated and displayed	Passed	-
61	TC-Jest-11	Customer Editing	Edit customer Eve	1. Mock axios.get and axios.put 2. Render <App /> 3. Click Edit button for Eve 4. Change Name or Email field 5. Click Update Customer button 6. Assert updated Eve is displayed	Eve in mockCustomers	Eve updated and displayed	Eve updated and displayed	Passed	-
62	TC-Jest-12	Customer Deletion	Delete customer Alice	1. Mock axios.get and axios.delete 2. Render <App /> 3. Click Delete button for Alice 4. Assert Alice is removed from display	Alice in mockCustomers	Alice removed from display	Alice removed from display	Passed	-
63	TC-Jest-13	Customer Deletion	Delete customer Bob	1. Mock axios.get and axios.delete 2. Render <App /> 3. Click Delete button for Bob 4. Assert Bob is removed from display	Bob in mockCustomers	Bob removed from display	Bob removed from display	Passed	-

Excel Link

[12575-Rishi-NUnit & Jest](#)

Git hub Link

[benhar-velankanni/RelevantzRIX2024 at 12575_Rishi](#)

THANK YOU.....